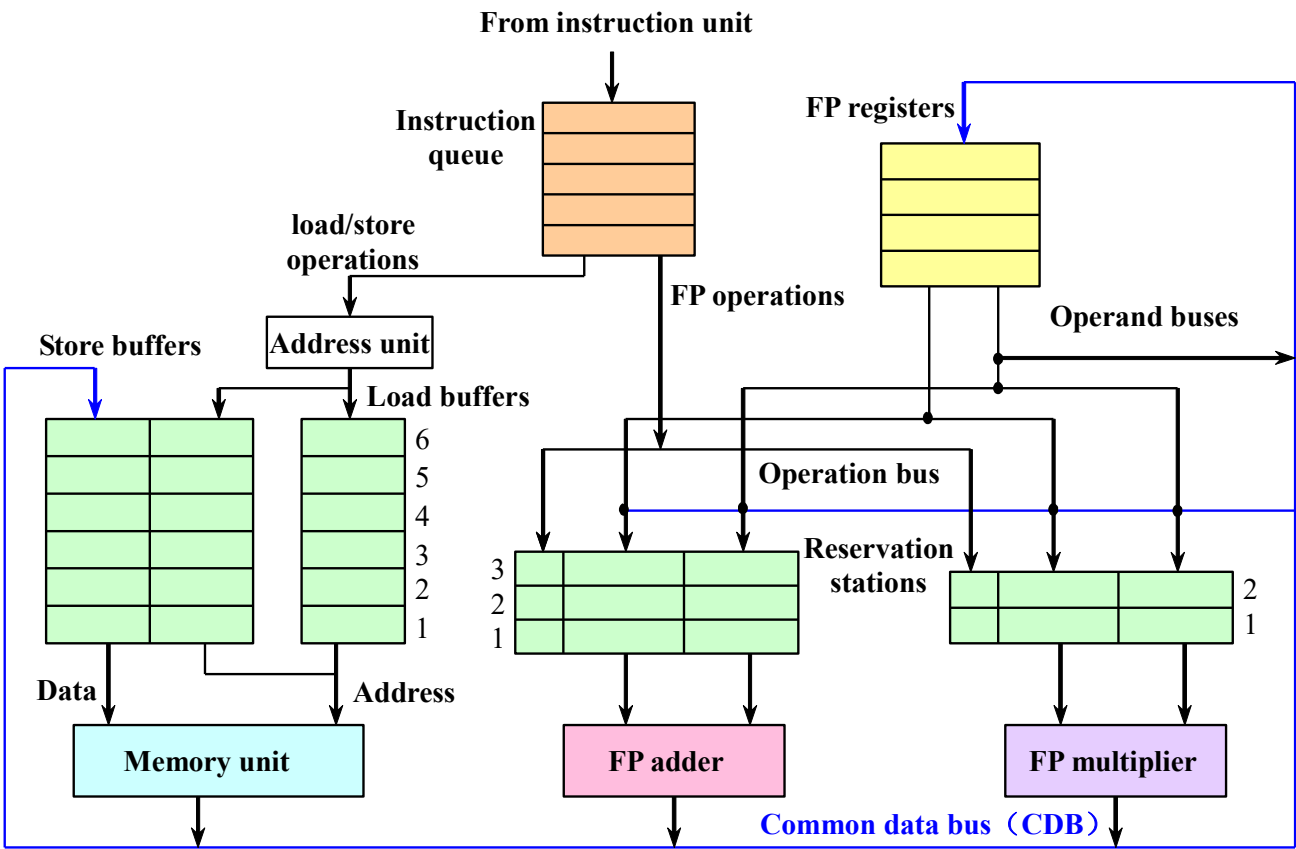


# Tomasulo's Approach



*The basic structure of a floating-point unit using Tomasulo's algorithm*

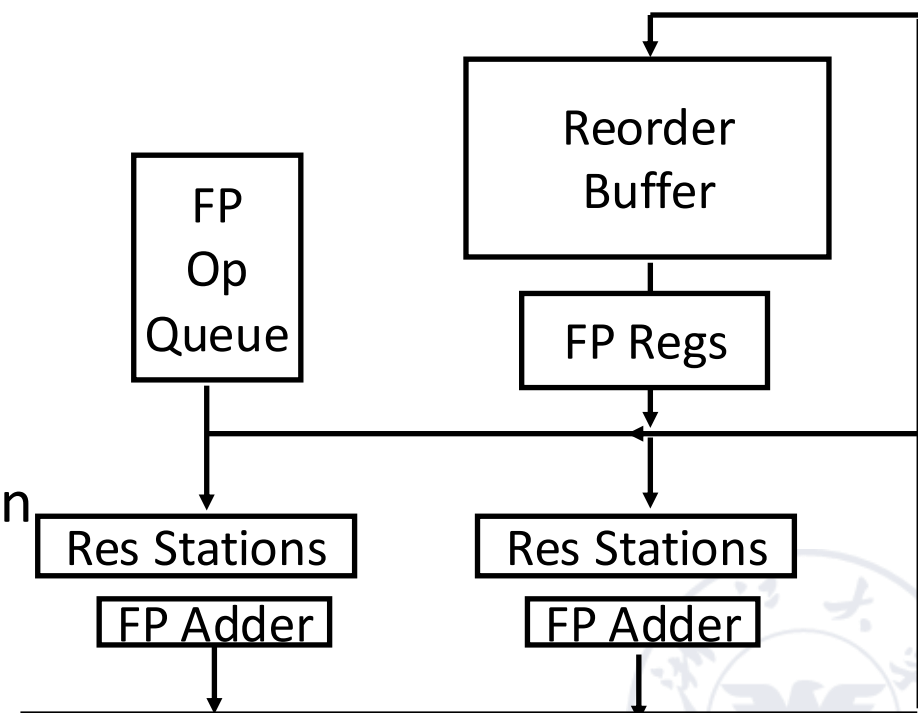


# Hardware-Based Speculation

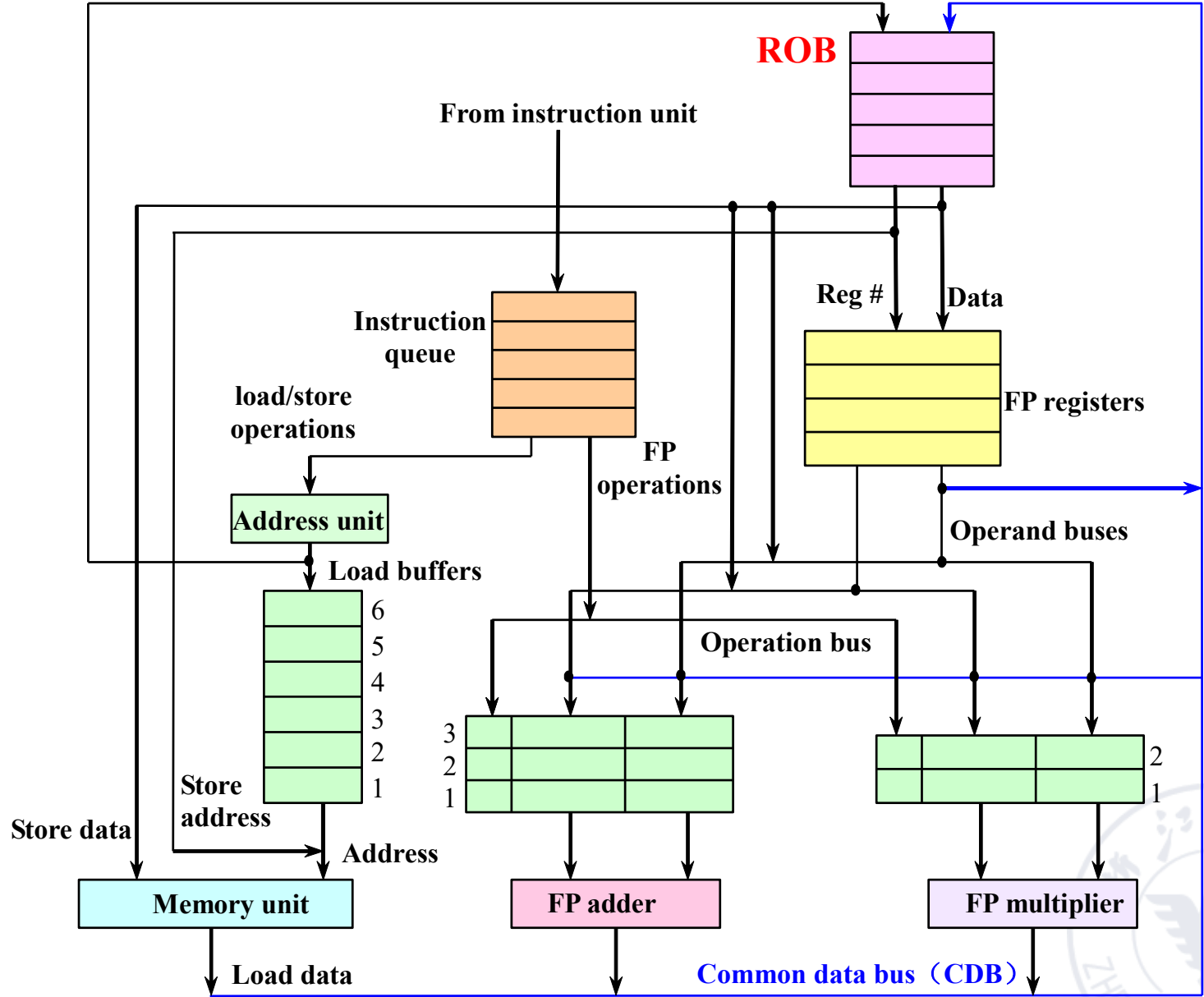
Cache for uncommitted instruction results:

3 fields: instruction type, destination address, value

1. When the program execution phase is completed, replace the value in RS with the number of ROB
2. Increase instruction submission stage
3. ROB provides the number of operations in the completion phase and the commit phase
4. Once the operand is submitted, the result is written to the register
5. In this way, when the prediction fails, it is easy to restore the inferred execution instruction, or when an exception occurs, it is easy to restore the state



The basic structure of a FP unit using Tomasulo's algorithm and extended to handle speculation.



# Hardware-Based Speculation

1. Issue—get instruction from FP Op Queue
2. Execution—operate on operands (EX)
3. Write result—finish execution (WB)
4. Commit—update register with reorder result



# Hardware-Based Speculation

- Hardware-based speculation combines three key ideas
  - dynamic branch prediction to choose which instructions to execute
  - speculation to allow the execution of instructions before the control dependences are resolved (with the ability to undo the effects of an incorrectly speculated sequence)
  - dynamic scheduling to deal with the scheduling of different combinations of basic blocks



# Hardware-Based Speculation

- WB
  - The ROB holds the result of an instruction between the time the operation associated with the instruction completes and the time the instruction commits
  - The ROB is a source of operands for instructions, just as the reservation stations provide operands in Tomasulo's algorithm.
- instruction commit
  - The key idea behind implementing speculation is to allow instructions to execute out of order but to force them to commit in order and to prevent any irrevocable action (such as updating state or taking an exception) until an instruction commits.
  - The reorder buffer (ROB) provides additional registers in the same way as the reservation stations in Tomasulo's algorithm extend the register set.



# Show what the status tables look like when the FMUL.D is ready to commit

|        |           |
|--------|-----------|
| FLD    | F6,34(R2) |
| FLD    | F2,45(R3) |
| FMUL.D | F0,F2,F4  |
| FSUB.D | F8,F6,F2  |
| FDIV.D | F10,F0,F6 |
| FADD.D | F6,F8,F2  |



# Hardware-Based Speculation

| Name  | Reservation Station |     |                 |                 |    |    |      |   |
|-------|---------------------|-----|-----------------|-----------------|----|----|------|---|
|       | Busy                | Op  | Vj              | Vk              | Qj | Qk | Dest | A |
| Add1  | no                  |     |                 |                 |    |    |      |   |
| Add2  | no                  |     |                 |                 |    |    |      |   |
| Add3  | no                  |     |                 |                 |    |    |      |   |
| Mult1 | no                  | MUL | Mem[45+Reg[R2]] | Regs[F4]        |    |    | #3   |   |
| Mult2 | yes                 | DIV |                 | Mem[34+Reg[R2]] | #3 |    | #5   |   |





| NO. | ROB  |                    |        |        |                  |
|-----|------|--------------------|--------|--------|------------------|
|     | Busy | Instruction        | Status | Object | Value            |
| 1   | no   | FLD F6, 34(R2)     | Commit | F6     | Mem[34+Regs[R2]] |
| 2   | no   | FLD F2, 45(R3)     | Commit | F2     | Mem[45+Regs[R3]] |
| 3   | yes  | FMUL.D F0, F2, F4  | WB     | F0     | #2 × Regs[F4]    |
| 4   | yes  | FSUB.D F8, F6, F2  | WB     | F8     | #1 - #2          |
| 5   | yes  | FDIV.D F10, F0, F6 | EX     | F10    |                  |
| 6   | yes  | FADD.D F6, F8, F2  | WB     | F6     | #4 + #2          |

| Name | Register Status |    |    |     |     |     |     |     |
|------|-----------------|----|----|-----|-----|-----|-----|-----|
|      | F0              | F2 | F4 | F6  | F8  | F10 | ... | F30 |
| ROB  | 3               |    |    | 6   | 4   | 5   |     |     |
| Busy | yes             | no | no | yes | yes | yes | ... | no  |



# Practice in Class

Suppose:

Add instruction needs 2 clock cycles. Multiply instruction needs 10 clock cycles.

Division instruction needs 40 clock cycles. LD instruction need 1 clock cycles.

|               |                    |
|---------------|--------------------|
| <b>FLD</b>    | <b>F6, 34 (R2)</b> |
| <b>FLD</b>    | <b>F2, 45 (R3)</b> |
| <b>FMUL.D</b> | <b>F0, F2, F4</b>  |
| <b>FSUB.D</b> | <b>F8, F2, F6</b>  |
| <b>FDIV.D</b> | <b>F10, F0, F6</b> |
| <b>FADD.D</b> | <b>F6, F8, F2</b>  |

How many cycles does it take to finish each instruction by hardware-based speculation?

# Hardware-Based Speculation

Suppose:

- Add instruction needs 2 clock cycles. Multiply instruction needs 10 clock cycles. Division instruction needs 40 clock cycles. LD instruction need 1 clock cycles.

FLD            F6, 34(R2)

FLD            F2, 45(R3)

FMUL.D       F0, F2, F4

FSUB.D       F8, F2, F8

FDIV.D       F10, F0, F6

FADD.D       F6, F8, F2

- How many cycles does it take to finish each instruction using Hardware-Based Speculation?



# Tomasulo with Reorder Buffer - Cycle 0

|       | Busy | Op | Vj | Vk | Qj | Qk | Dest | A |
|-------|------|----|----|----|----|----|------|---|
| Add1  | No   |    |    |    |    |    |      |   |
| Add2  | No   |    |    |    |    |    |      |   |
| Add3  | No   |    |    |    |    |    |      |   |
| Mult1 | No   |    |    |    |    |    |      |   |
| Mult2 | No   |    |    |    |    |    |      |   |

Reservation Stations

|   | Busy | Instruction | Status | Object | Value |
|---|------|-------------|--------|--------|-------|
| 1 | no   |             |        |        |       |
| 2 | no   |             |        |        |       |
| 3 | no   |             |        |        |       |
| 4 | no   |             |        |        |       |
| 5 | no   |             |        |        |       |
| 6 | no   |             |        |        |       |

|       | Busy | Instruction |
|-------|------|-------------|
| Load1 |      |             |
| Load2 |      |             |
| Load3 |      |             |

Reorder Buffer

|      | F0 | F2 | F4 | F6 | F8 | F10 | ... | F30 |
|------|----|----|----|----|----|-----|-----|-----|
| ROB  |    |    |    |    |    |     |     |     |
| Busy | no | no | no | no | no | no  | ... | no  |



# Tomasulo with Reorder Buffer - Cycle 1

|       | Busy | Op | Vj | Vk | Qj | Qk | Dest | A |
|-------|------|----|----|----|----|----|------|---|
| Add1  | No   |    |    |    |    |    |      |   |
| Add2  | No   |    |    |    |    |    |      |   |
| Add3  | No   |    |    |    |    |    |      |   |
| Mult1 | No   |    |    |    |    |    |      |   |
| Mult2 | No   |    |    |    |    |    |      |   |

**Reservation Stations**

|   | Busy | Instruction    | Status | Dest | Value |
|---|------|----------------|--------|------|-------|
| 1 | Yes  | FLD F6, 34(R2) | Issue  | F6   |       |
| 2 | no   |                |        |      |       |
| 3 | no   |                |        |      |       |
| 4 | no   |                |        |      |       |
| 5 | no   |                |        |      |       |
| 6 | no   |                |        |      |       |

|       | Busy | Instruction   |
|-------|------|---------------|
| Load1 | Yes  | 34 + Regs[R2] |
| Load2 |      |               |
| Load3 |      |               |

**Reorder Buffer**

|      | F0 | F2 | F4 | F6  | F8 | F10 | ... | F30 |
|------|----|----|----|-----|----|-----|-----|-----|
| ROB  |    |    |    | #1  |    |     |     |     |
| Busy | no | no | no | Yes | no | no  | ... | no  |



# Tomasulo with Reorder Buffer - Cycle 2

|       | Busy | Op | Vj | Vk | Qj | Qk | Dest | A |
|-------|------|----|----|----|----|----|------|---|
| Add1  | No   |    |    |    |    |    |      |   |
| Add2  | No   |    |    |    |    |    |      |   |
| Add3  | No   |    |    |    |    |    |      |   |
| Mult1 | No   |    |    |    |    |    |      |   |
| Mult2 | No   |    |    |    |    |    |      |   |

**Reservation Stations**

|   | Busy | Instruction    | Status | Dest | Value      |
|---|------|----------------|--------|------|------------|
| 1 | Yes  | FLD F6, 34(R2) | Issue  | F6   | Mem[load1] |
| 2 | Yes  | FLD F2, 45(R3) | Issue  | F2   |            |
| 3 | no   |                |        |      |            |
| 4 | no   |                |        |      |            |
| 5 | no   |                |        |      |            |
| 6 | no   |                |        |      |            |

|       | Busy | Instruction   |
|-------|------|---------------|
| Load1 | Yes  | 34 + Regs[R2] |
| Load2 | Yes  | 45 + Regs[R3] |
| Load3 |      |               |

**Reorder Buffer**

|      | F0 | F2  | F4 | F6  | F8 | F10 | ... | F30 |
|------|----|-----|----|-----|----|-----|-----|-----|
| ROB  |    | #2  |    | #1  |    |     |     |     |
| Busy | no | Yes | no | Yes | no | no  | ... | no  |



# Tomasulo with Reorder Buffer - Cycle 3

|       | Busy | Op  | Vj | Vk       | Qj | Qk | Dest | A |
|-------|------|-----|----|----------|----|----|------|---|
| Add1  | No   |     |    |          |    |    |      |   |
| Add2  | No   |     |    |          |    |    |      |   |
| Add3  | No   |     |    |          |    |    |      |   |
| Mult1 | Yes  | Mul |    | Regs[F4] | #2 |    | #3   |   |
| Mult2 | No   |     |    |          |    |    |      |   |

**Reservation Stations**

|   | Busy | Instruction       | Status | Dest | Value      |
|---|------|-------------------|--------|------|------------|
| 1 | Yes  | FLD F6, 34(R2)    | Ex1    | F6   | Mem[load1] |
| 2 | Yes  | FLD F2, 45(R3)    | Issue  | F2   | Mem[load2] |
| 3 | Yes  | FMUL.D F0, F2, F4 | Issue  | F0   |            |
| 4 | no   |                   |        |      |            |
| 5 | no   |                   |        |      |            |
| 6 | no   |                   |        |      |            |

|       | Busy | Instruction   |
|-------|------|---------------|
| Load1 | Yes  | 34 + Regs[R2] |
| Load2 | Yes  | 45 + Regs[R3] |
| Load3 |      |               |

**Reorder Buffer**

|      | F0  | F2  | F4 | F6  | F8 | F10 | ... | F30 |
|------|-----|-----|----|-----|----|-----|-----|-----|
| ROB  | #3  | #2  |    | #1  |    |     |     |     |
| Busy | Yes | Yes | no | Yes | no | no  | ... | no  |



# Tomasulo with Reorder Buffer - Cycle 4

|       | Busy | Op  | Vj               | Vk               | Qj | Qk | Dest | A |
|-------|------|-----|------------------|------------------|----|----|------|---|
| Add1  | Yes  | Sub | Regs[F6]         | Mem[45+Regs[R3]] |    |    | #4   |   |
| Add2  | No   |     |                  |                  |    |    |      |   |
| Add3  | No   |     |                  |                  |    |    |      |   |
| Mult1 | Yes  | Mul | Mem(45+Regs[R3]) | Regs[F4]         |    |    | #3   |   |
| Mult2 | No   |     |                  |                  |    |    |      |   |

**Reservation Stations**

|   | Busy | Instruction       | Status | Dest | Value      |
|---|------|-------------------|--------|------|------------|
| 1 | Yes  | FLD F6, 34(R2)    | write  | F6   | Mem[load1] |
| 2 | Yes  | FLD F2, 45(R3)    | Ex1    | F2   | Mem[load2] |
| 3 | Yes  | FMUL.D F0, F2, F4 | Issue  | F0   |            |
| 4 | Yes  | FSUB.D F8, F6, F2 | Issue  | F8   |            |
| 5 | no   |                   |        |      |            |
| 6 | no   |                   |        |      |            |

|       | Busy | Instruction   |
|-------|------|---------------|
| Load1 | Yes  | 34 + Regs[R2] |
| Load2 | Yes  | 45 + Regs[R3] |
| Load3 |      |               |

**Reorder Buffer**

|      | F0  | F2  | F4 | F6  | F8  | F10 | ... | F30 |
|------|-----|-----|----|-----|-----|-----|-----|-----|
| ROB  | #3  | #2  |    | #1  | #4  |     |     |     |
| Busy | Yes | Yes | no | Yes | Yes | no  | ... | no  |





# Tomasulo with Reorder Buffer - Cycle 5

|       | Busy | Op  | Vj               | Vk               | Qj | Qk | Dest | A |
|-------|------|-----|------------------|------------------|----|----|------|---|
| Add1  | Yes  | Sub | Regs[F6]         | Mem[45+Regs[R3]] |    |    | #4   |   |
| Add2  | No   |     |                  |                  |    |    |      |   |
| Add3  | No   |     |                  |                  |    |    |      |   |
| Mult1 | Yes  | Mul | Mem(45+Regs[R3]) | Regs[F4]         |    |    | #3   |   |
| Mult2 | Yes  | Div |                  | Regs[F6]         | #3 |    | #5   |   |

**Reservation Stations**

|   | Busy | Instruction        | Status | Dest | Value      |
|---|------|--------------------|--------|------|------------|
| 1 | no   | FLD F6, 34(R2)     | commit | F6   | Mem[load1] |
| 2 | Yes  | FLD F2, 45(R3)     | write  | F2   | Mem[load2] |
| 3 | Yes  | FMUL.D F0, F2, F4  | Issue  | F0   |            |
| 4 | Yes  | FSUB.D F8, F6, F2  | Issue  | F8   |            |
| 5 | Yes  | FDIV.D F10, F0, F6 | Issue  | F10  |            |
| 6 | no   |                    |        |      |            |

|       | Busy | Instruction |
|-------|------|-------------|
| Load1 | no   |             |
| Load2 | no   |             |
| Load3 |      |             |

**Reorder Buffer**

|      | F0  | F2  | F4 | F6 | F8  | F10 | ... | F30 |
|------|-----|-----|----|----|-----|-----|-----|-----|
| ROB  | #3  | #2  |    |    | #4  | #5  |     |     |
| Busy | Yes | Yes | no | no | Yes | Yes | ... | no  |



# Tomasulo with Reorder Buffer - Cycle 6

|       | Busy | Op  | Vj               | Vk               | Qj | Qk | Dest | A |
|-------|------|-----|------------------|------------------|----|----|------|---|
| Add1  | Yes  | Sub | Regs[F6]         | Mem[45+Regs[R3]] |    |    | #4   |   |
| Add2  | Yes  | Add |                  | Regs[F2]         | #4 |    | #6   |   |
| Add3  | No   |     |                  |                  |    |    |      |   |
| Mult1 | Yes  | Mul | Mem(45+Regs[R3]) | Regs[F4]         |    |    | #3   |   |
| Mult2 | Yes  | Div |                  | Regs[F6]         | #3 |    | #5   |   |

**Reservation Stations**

|   | Busy | Instruction        | Status | Dest | Value      |       | Busy | Instruction |
|---|------|--------------------|--------|------|------------|-------|------|-------------|
| 1 | no   | FLD F6, 34(R2)     | commit | F6   | Mem[load1] | Load1 | no   |             |
| 2 | no   | FLD F2, 45(R3)     | commit | F2   | Mem[load2] | Load2 | no   |             |
| 3 | Yes  | FMUL.D F0, F2, F4  | Ex1    | F0   |            | Load3 |      |             |
| 4 | Yes  | FSUB.D F8, F6, F2  | Ex1    | F8   |            |       |      |             |
| 5 | Yes  | FDIV.D F10, F0, F6 | Issue  | F10  |            |       |      |             |
| 6 | Yes  | FADD.D F6, F8, F2  | Issue  | F6   |            |       |      |             |

**Reorder Buffer**

|      | F0  | F2 | F4 | F6  | F8  | F10 | ... | F30 |
|------|-----|----|----|-----|-----|-----|-----|-----|
| ROB  | #3  |    |    | #6  | #4  | #5  |     |     |
| Busy | Yes | no | no | Yes | Yes | Yes | ... | no  |



# Tomasulo with Reorder Buffer - Cycle 7

|       | Busy | Op  | Vj               | Vk               | Qj | Qk | Dest | A |
|-------|------|-----|------------------|------------------|----|----|------|---|
| Add1  | No   | Sub | Regs[F6]         | Mem[45+Regs[R3]] |    |    | #4   |   |
| Add2  | Yes  | Add | #4               | Regs[F2]         |    |    | #6   |   |
| Add3  | No   |     |                  |                  |    |    |      |   |
| Mult1 | Yes  | Mul | Mem(45+Regs[R3]) | Regs[F4]         |    |    | #3   |   |
| Mult2 | Yes  | Div |                  | Regs[F6]         | #3 |    | #5   |   |

**Reservation Stations**

|   | Busy | Instruction        | Status | Dest | Value      |
|---|------|--------------------|--------|------|------------|
| 1 | no   | FLD F6, 34(R2)     | commit | F6   | Mem[load1] |
| 2 | no   | FLD F2, 45(R3)     | commit | F2   | Mem[load2] |
| 3 | Yes  | FMUL.D F0, F2, F4  | Ex2    | F0   |            |
| 4 | Yes  | FSUB.D F8, F6, F2  | Ex2    | F8   | F6 - #2    |
| 5 | Yes  | FDIV.D F10, F0, F6 | Issue  | F10  |            |
| 6 | Yes  | FADD.D F6, F8, F2  | Issue  | F6   |            |

|       | Busy | Instruction |
|-------|------|-------------|
| Load1 | no   |             |
| Load2 | no   |             |
| Load3 |      |             |

**Reorder Buffer**

|      | F0  | F2 | F4 | F6  | F8  | F10 | ... | F30 |
|------|-----|----|----|-----|-----|-----|-----|-----|
| ROB  | #3  |    |    | #6  | #4  | #5  |     |     |
| Busy | Yes | no | no | Yes | Yes | Yes | ... | no  |



# Tomasulo with Reorder Buffer - Cycle 8

|       | Busy | Op  | Vj               | Vk               | Qj | Qk | Dest | A |
|-------|------|-----|------------------|------------------|----|----|------|---|
| Add1  | No   | Sub | Regs[F6]         | Mem[45+Regs[R3]] |    |    | #4   |   |
| Add2  | Yes  | Add | #4               | Regs[F2]         |    |    | #6   |   |
| Add3  | No   |     |                  |                  |    |    |      |   |
| Mult1 | Yes  | Mul | Mem(45+Regs[R3]) | Regs[F4]         |    |    | #3   |   |
| Mult2 | Yes  | Div |                  | Regs[F6]         | #3 |    | #5   |   |

**Reservation Stations**

|   | Busy | Instruction        | Status | Dest | Value      |
|---|------|--------------------|--------|------|------------|
| 1 | no   | FLD F6, 34(R2)     | commit | F6   | Mem[load1] |
| 2 | no   | FLD F2, 45(R3)     | commit | F2   | Mem[load2] |
| 3 | Yes  | FMUL.D F0, F2, F4  | Ex3    | F0   |            |
| 4 | Yes  | FSUB.D F8, F6, F2  | write  | F8   | F6 - #2    |
| 5 | Yes  | FDIV.D F10, F0, F6 | Issue  | F10  |            |
| 6 | Yes  | FADD.D F6, F8, F2  | Issue  | F6   |            |

|       | Busy | Instruction |
|-------|------|-------------|
| Load1 | no   |             |
| Load2 | no   |             |
| Load3 |      |             |

**Reorder Buffer**

|      | F0  | F2 | F4 | F6  | F8  | F10 | ... | F30 |
|------|-----|----|----|-----|-----|-----|-----|-----|
| ROB  | #3  |    |    | #6  | #4  | #5  |     |     |
| Busy | Yes | no | no | Yes | Yes | Yes | ... | no  |



# Tomasulo with Reorder Buffer - Cycle 9

|       | Busy | Op  | Vj               | Vk               | Qj | Qk | Dest | A |
|-------|------|-----|------------------|------------------|----|----|------|---|
| Add1  | No   | Sub | Regs[F6]         | Mem[45+Regs[R3]] |    |    | #4   |   |
| Add2  | Yes  | Add | #4               | Regs[F2]         |    |    | #6   |   |
| Add3  | No   |     |                  |                  |    |    |      |   |
| Mult1 | Yes  | Mul | Mem(45+Regs[R3]) | Regs[F4]         |    |    | #3   |   |
| Mult2 | Yes  | Div |                  | Regs[F6]         | #3 |    | #5   |   |

**Reservation Stations**

|   | Busy | Instruction        | Status | Dest | Value      |
|---|------|--------------------|--------|------|------------|
| 1 | no   | FLD F6, 34(R2)     | commit | F6   | Mem[load1] |
| 2 | no   | FLD F2, 45(R3)     | commit | F2   | Mem[load2] |
| 3 | Yes  | FMUL.D F0, F2, F4  | Ex4    | F0   |            |
| 4 | Yes  | FSUB.D F8, F6, F2  | write  | F8   | F6 - #2    |
| 5 | Yes  | FDIV.D F10, F0, F6 | Issue  | F10  |            |
| 6 | Yes  | FADD.D F6, F8, F2  | Ex1    | F6   | #4 + F2    |

|       | Busy | Instruction |
|-------|------|-------------|
| Load1 | no   |             |
| Load2 | no   |             |
| Load3 |      |             |

**Reorder Buffer**

|      | F0  | F2 | F4 | F6  | F8  | F10 | ... | F30 |
|------|-----|----|----|-----|-----|-----|-----|-----|
| ROB  | #3  |    |    | #6  | #4  | #5  |     |     |
| Busy | Yes | no | no | Yes | Yes | Yes | ... | no  |



# Tomasulo with Reorder Buffer - Cycle 10

|       | Busy | Op  | Vj               | Vk               | Qj | Qk | Dest | A |
|-------|------|-----|------------------|------------------|----|----|------|---|
| Add1  | No   | Sub | Regs[F6]         | Mem[45+Regs[R3]] |    |    | #4   |   |
| Add2  | Yes  | Add | #4               | Regs[F2]         |    |    | #6   |   |
| Add3  | No   |     |                  |                  |    |    |      |   |
| Mult1 | Yes  | Mul | Mem(45+Regs[R3]) | Regs[F4]         |    |    | #3   |   |
| Mult2 | Yes  | Div |                  | Regs[F6]         | #3 |    | #5   |   |

**Reservation Stations**

|   | Busy | Instruction        | Status | Dest | Value      |
|---|------|--------------------|--------|------|------------|
| 1 | no   | FLD F6, 34(R2)     | commit | F6   | Mem[load1] |
| 2 | no   | FLD F2, 45(R3)     | commit | F2   | Mem[load2] |
| 3 | Yes  | FMUL.D F0, F2, F4  | Ex5    | F0   |            |
| 4 | Yes  | FSUB.D F8, F6, F2  | write  | F8   | F6 - #2    |
| 5 | Yes  | FDIV.D F10, F0, F6 | Issue  | F10  |            |
| 6 | Yes  | FADD.D F6, F8, F2  | Ex2    | F6   | #4 + F2    |

|       | Busy | Instruction |
|-------|------|-------------|
| Load1 | no   |             |
| Load2 | no   |             |
| Load3 |      |             |

**Reorder Buffer**

|      | F0  | F2 | F4 | F6  | F8  | F10 | ... | F30 |
|------|-----|----|----|-----|-----|-----|-----|-----|
| ROB  | #3  |    |    | #6  | #4  | #5  |     |     |
| Busy | Yes | no | no | Yes | Yes | Yes | ... | no  |



# Tomasulo with Reorder Buffer - Cycle 11

|       | Busy | Op  | Vj               | Vk               | Qj | Qk | Dest | A |
|-------|------|-----|------------------|------------------|----|----|------|---|
| Add1  | No   | Sub | Regs[F6]         | Mem[45+Regs[R3]] |    |    | #4   |   |
| Add2  | No   | Add | #4               | Regs[F2]         |    |    | #6   |   |
| Add3  | No   |     |                  |                  |    |    |      |   |
| Mult1 | Yes  | Mul | Mem(45+Regs[R3]) | Regs[F4]         |    |    | #3   |   |
| Mult2 | Yes  | Div |                  | Regs[F6]         | #3 |    | #5   |   |

**Reservation Stations**

|   | Busy | Instruction        | Status | Dest | Value      |
|---|------|--------------------|--------|------|------------|
| 1 | no   | FLD F6, 34(R2)     | commit | F6   | Mem[load1] |
| 2 | no   | FLD F2, 45(R3)     | commit | F2   | Mem[load2] |
| 3 | Yes  | FMUL.D F0, F2, F4  | Ex6    | F0   |            |
| 4 | Yes  | FSUB.D F8, F6, F2  | write  | F8   | F6 - #2    |
| 5 | Yes  | FDIV.D F10, F0, F6 | Issue  | F10  |            |
| 6 | Yes  | FADD.D F6, F8, F2  | write  | F6   | #4 + F2    |

|       | Busy | Instruction |
|-------|------|-------------|
| Load1 | no   |             |
| Load2 | no   |             |
| Load3 |      |             |

**Reorder Buffer**

|      | F0  | F2 | F4 | F6  | F8  | F10 | ... | F30 |
|------|-----|----|----|-----|-----|-----|-----|-----|
| ROB  | #3  |    |    | #6  | #4  | #5  |     |     |
| Busy | Yes | no | no | Yes | Yes | Yes | ... | no  |



# Tomasulo with Reorder Buffer - Cycle 12

|       | Busy | Op  | Vj               | Vk               | Qj | Qk | Dest | A |
|-------|------|-----|------------------|------------------|----|----|------|---|
| Add1  | No   | Sub | Regs[F6]         | Mem[45+Regs[R3]] |    |    | #4   |   |
| Add2  | No   | Add | #4               | Regs[F2]         |    |    | #6   |   |
| Add3  | No   |     |                  |                  |    |    |      |   |
| Mult1 | Yes  | Mul | Mem(45+Regs[R3]) | Regs[F4]         |    |    | #3   |   |
| Mult2 | Yes  | Div |                  | Regs[F6]         | #3 |    | #5   |   |

**Reservation Stations**

|   | Busy | Instruction        | Status | Dest | Value      |
|---|------|--------------------|--------|------|------------|
| 1 | no   | FLD F6, 34(R2)     | commit | F6   | Mem[load1] |
| 2 | no   | FLD F2, 45(R3)     | commit | F2   | Mem[load2] |
| 3 | Yes  | FMUL.D F0, F2, F4  | Ex7    | F0   |            |
| 4 | Yes  | FSUB.D F8, F6, F2  | write  | F8   | F6 - #2    |
| 5 | Yes  | FDIV.D F10, F0, F6 | Issue  | F10  |            |
| 6 | Yes  | FADD.D F6, F8, F2  | write  | F6   | #4 + F2    |

|       | Busy | Instruction |
|-------|------|-------------|
| Load1 | no   |             |
| Load2 | no   |             |
| Load3 |      |             |

**Reorder Buffer**

|      | F0  | F2 | F4 | F6  | F8  | F10 | ... | F30 |
|------|-----|----|----|-----|-----|-----|-----|-----|
| ROB  | #3  |    |    | #6  | #4  | #5  |     |     |
| Busy | Yes | no | no | Yes | Yes | Yes | ... | no  |





# Tomasulo with Reorder Buffer - Cycle 13

|       | Busy | Op  | Vj               | Vk               | Qj | Qk | Dest | A |
|-------|------|-----|------------------|------------------|----|----|------|---|
| Add1  | No   | Sub | Regs[F6]         | Mem[45+Regs[R3]] |    |    | #4   |   |
| Add2  | No   | Add | #4               | Regs[F2]         |    |    | #6   |   |
| Add3  | No   |     |                  |                  |    |    |      |   |
| Mult1 | Yes  | Mul | Mem(45+Regs[R3]) | Regs[F4]         |    |    | #3   |   |
| Mult2 | Yes  | Div |                  | Regs[F6]         | #3 |    | #5   |   |

**Reservation Stations**

|   | Busy | Instruction        | Status | Dest | Value      |
|---|------|--------------------|--------|------|------------|
| 1 | no   | FLD F6, 34(R2)     | commit | F6   | Mem[load1] |
| 2 | no   | FLD F2, 45(R3)     | commit | F2   | Mem[load2] |
| 3 | Yes  | FMUL.D F0, F2, F4  | Ex8    | F0   |            |
| 4 | Yes  | FSUB.D F8, F6, F2  | write  | F8   | F6 - #2    |
| 5 | Yes  | FDIV.D F10, F0, F6 | Issue  | F10  |            |
| 6 | Yes  | FADD.D F6, F8, F2  | write  | F6   | #4 + F2    |

|       | Busy | Instruction |
|-------|------|-------------|
| Load1 | no   |             |
| Load2 | no   |             |
| Load3 |      |             |

**Reorder Buffer**

|      | F0  | F2 | F4 | F6  | F8  | F10 | ... | F30 |
|------|-----|----|----|-----|-----|-----|-----|-----|
| ROB  | #3  |    |    | #6  | #4  | #5  |     |     |
| Busy | Yes | no | no | Yes | Yes | Yes | ... | no  |



# Tomasulo with Reorder Buffer - Cycle 14

|       | Busy | Op  | Vj               | Vk               | Qj | Qk | Dest | A |
|-------|------|-----|------------------|------------------|----|----|------|---|
| Add1  | No   | Sub | Regs[F6]         | Mem[45+Regs[R3]] |    |    | #4   |   |
| Add2  | No   | Add | #4               | Regs[F2]         |    |    | #6   |   |
| Add3  | No   |     |                  |                  |    |    |      |   |
| Mult1 | Yes  | Mul | Mem(45+Regs[R3]) | Regs[F4]         |    |    | #3   |   |
| Mult2 | Yes  | Div | #2 * Regs[F4]    | Regs[F6]         | #3 |    | #5   |   |

**Reservation Stations**

|   | Busy | Instruction        | Status | Dest | Value         |
|---|------|--------------------|--------|------|---------------|
| 1 | no   | FLD F6, 34(R2)     | commit | F6   | Mem[load1]    |
| 2 | no   | FLD F2, 45(R3)     | commit | F2   | Mem[load2]    |
| 3 | Yes  | FMUL.D F0, F2, F4  | Ex9    | F0   | #2 * Regs[F4] |
| 4 | Yes  | FSUB.D F8, F6, F2  | write  | F8   | F6 - #2       |
| 5 | Yes  | FDIV.D F10, F0, F6 | Issue  | F10  |               |
| 6 | Yes  | FADD.D F6, F8, F2  | write  | F6   | #4 + F2       |

|       | Busy | Instruction |
|-------|------|-------------|
| Load1 | no   |             |
| Load2 | no   |             |
| Load3 |      |             |

**Reorder Buffer**

|      | F0  | F2 | F4 | F6  | F8  | F10 | ... | F30 |
|------|-----|----|----|-----|-----|-----|-----|-----|
| ROB  | #3  |    |    | #6  | #4  | #5  |     |     |
| Busy | Yes | no | no | Yes | Yes | Yes | ... | no  |



# Tomasulo with Reorder Buffer - Cycle 15

|       | Busy | Op  | Vj               | Vk               | Qj | Qk | Dest | A |
|-------|------|-----|------------------|------------------|----|----|------|---|
| Add1  | No   | Sub | Regs[F6]         | Mem[45+Regs[R3]] |    |    | #4   |   |
| Add2  | No   | Add | #4               | Regs[F2]         |    |    | #6   |   |
| Add3  | No   |     |                  |                  |    |    |      |   |
| Mult1 | Yes  | Mul | Mem(45+Regs[R3]) | Regs[F4]         |    |    | #3   |   |
| Mult2 | Yes  | Div | #2 * Regs[F4]    | Regs[F6]         | #3 |    | #5   |   |

**Reservation Stations**

|   | Busy | Instruction        | Status | Dest | Value         |
|---|------|--------------------|--------|------|---------------|
| 1 | no   | FLD F6, 34(R2)     | commit | F6   | Mem[load1]    |
| 2 | no   | FLD F2, 45(R3)     | commit | F2   | Mem[load2]    |
| 3 | Yes  | FMUL.D F0, F2, F4  | Ex10   | F0   | #2 * Regs[F4] |
| 4 | Yes  | FSUB.D F8, F6, F2  | write  | F8   | F6 - #2       |
| 5 | Yes  | FDIV.D F10, F0, F6 | Issue  | F10  |               |
| 6 | Yes  | FADD.D F6, F8, F2  | write  | F6   | #4 + F2       |

|       | Busy | Instruction |
|-------|------|-------------|
| Load1 | no   |             |
| Load2 | no   |             |
| Load3 |      |             |

**Reorder Buffer**

|      | F0  | F2 | F4 | F6  | F8  | F10 | ... | F30 |
|------|-----|----|----|-----|-----|-----|-----|-----|
| ROB  | #3  |    |    | #6  | #4  | #5  |     |     |
| Busy | Yes | no | no | Yes | Yes | Yes | ... | no  |



# Tomasulo with Reorder Buffer - Cycle 16

|       | Busy | Op  | Vj               | Vk               | Qj | Qk | Dest | A |
|-------|------|-----|------------------|------------------|----|----|------|---|
| Add1  | No   | Sub | Regs[F6]         | Mem[45+Regs[R3]] |    |    | #4   |   |
| Add2  | No   | Add | #4               | Regs[F2]         |    |    | #6   |   |
| Add3  | No   |     |                  |                  |    |    |      |   |
| Mult1 | No   | Mul | Mem(45+Regs[R3]) | Regs[F4]         |    |    | #3   |   |
| Mult2 | Yes  | Div | #2 * Regs[F4]    | Regs[F6]         | #3 |    | #5   |   |

**Reservation Stations**

|   | Busy | Instruction        | Status | Dest | Value         |
|---|------|--------------------|--------|------|---------------|
| 1 | no   | FLD F6, 34(R2)     | commit | F6   | Mem[load1]    |
| 2 | no   | FLD F2, 45(R3)     | commit | F2   | Mem[load2]    |
| 3 | Yes  | FMUL.D F0, F2, F4  | write  | F0   | #2 * Regs[F4] |
| 4 | Yes  | FSUB.D F8, F6, F2  | write  | F8   | F6 - #2       |
| 5 | Yes  | FDIV.D F10, F0, F6 | Issue  | F10  |               |
| 6 | Yes  | FADD.D F6, F8, F2  | write  | F6   | #4 + F2       |

|       | Busy | Instruction |
|-------|------|-------------|
| Load1 | no   |             |
| Load2 | no   |             |
| Load3 |      |             |

**Reorder Buffer**

|      | F0  | F2 | F4 | F6  | F8  | F10 | ... | F30 |
|------|-----|----|----|-----|-----|-----|-----|-----|
| ROB  | #3  |    |    | #6  | #4  | #5  |     |     |
| Busy | Yes | no | no | Yes | Yes | Yes | ... | no  |



# Tomasulo with Reorder Buffer - Cycle 17

|       | Busy | Op  | Vj            | Vk               | Qj | Qk | Dest | A |
|-------|------|-----|---------------|------------------|----|----|------|---|
| Add1  | No   | Sub | Regs[F6]      | Mem[45+Regs[R3]] |    |    | #4   |   |
| Add2  | No   | Add | #4            | Regs[F2]         |    |    | #6   |   |
| Add3  | No   |     |               |                  |    |    |      |   |
| Mult1 | No   |     |               |                  |    |    |      |   |
| Mult2 | Yes  | Div | #2 * Regs[F4] | Regs[F6]         | #3 |    | #5   |   |

**Reservation Stations**

|   | Busy | Instruction        | Status | Dest | Value         |
|---|------|--------------------|--------|------|---------------|
| 1 | no   | FLD F6, 34(R2)     | commit | F6   | Mem[load1]    |
| 2 | no   | FLD F2, 45(R3)     | commit | F2   | Mem[load2]    |
| 3 | no   | FMUL.D F0, F2, F4  | commit | F0   | #2 * Regs[F4] |
| 4 | Yes  | FSUB.D F8, F6, F2  | write  | F8   | F6 - #2       |
| 5 | Yes  | FDIV.D F10, F0, F6 | Ex1    | F10  |               |
| 6 | Yes  | FADD.D F6, F8, F2  | write  | F6   | #4 + F2       |

|       | Busy | Instruction |
|-------|------|-------------|
| Load1 | no   |             |
| Load2 | no   |             |
| Load3 |      |             |

**Reorder Buffer**

**Need 39 more EX cycles for DIV to finish**

|      | F0 | F2 | F4 | F6  | F8  | F10 | ... | F30 |
|------|----|----|----|-----|-----|-----|-----|-----|
| ROB  |    |    |    | #6  | #4  | #5  |     |     |
| Busy | no | no | no | Yes | Yes | Yes | ... | no  |



# Tomasulo with Reorder Buffer - Cycle 18

|       | Busy | Op  | Vj            | Vk       | Qj | Qk | Dest | A |
|-------|------|-----|---------------|----------|----|----|------|---|
| Add1  | No   |     |               |          |    |    |      |   |
| Add2  | No   | Add | #4            | Regs[F2] |    |    | #6   |   |
| Add3  | No   |     |               |          |    |    |      |   |
| Mult1 | No   |     |               |          |    |    |      |   |
| Mult2 | Yes  | Div | #2 * Regs[F4] | Regs[F6] | #3 |    | #5   |   |

**Reservation Stations**

|   | Busy | Instruction        | Status | Dest | Value         |
|---|------|--------------------|--------|------|---------------|
| 1 | no   | FLD F6, 34(R2)     | commit | F6   | Mem[load1]    |
| 2 | no   | FLD F2, 45(R3)     | commit | F2   | Mem[load2]    |
| 3 | no   | FMUL.D F0, F2, F4  | commit | F0   | #2 * Regs[F4] |
| 4 | no   | FSUB.D F8, F6, F2  | commit | F8   | F6 - #2       |
| 5 | Yes  | FDIV.D F10, F0, F6 | Ex2    | F10  |               |
| 6 | Yes  | FADD.D F6, F8, F2  | write  | F6   | #4 + F2       |

|       | Busy | Instruction |
|-------|------|-------------|
| Load1 | no   |             |
| Load2 | no   |             |
| Load3 |      |             |

**Reorder Buffer**

**Need 38 more EX cycles for DIV to finish**

|      | F0 | F2 | F4 | F6  | F8 | F10 | ... | F30 |
|------|----|----|----|-----|----|-----|-----|-----|
| ROB  |    |    |    | #6  |    | #5  |     |     |
| Busy | no | no | no | Yes | no | Yes | ... | no  |



# Tomasulo with Reorder Buffer - Cycle 56

|       | Busy | Op  | Vj            | Vk       | Qj | Qk | Dest | A |
|-------|------|-----|---------------|----------|----|----|------|---|
| Add1  | No   |     |               |          |    |    |      |   |
| Add2  | No   | Add | #4            | Regs[F2] |    |    | #6   |   |
| Add3  | No   |     |               |          |    |    |      |   |
| Mult1 | No   |     |               |          |    |    |      |   |
| Mult2 | Yes  | Div | #2 * Regs[F4] | Regs[F6] | #3 |    | #5   |   |

**Reservation Stations**

|   | Busy | Instruction        | Status | Dest | Value         |
|---|------|--------------------|--------|------|---------------|
| 1 | no   | FLD F6, 34(R2)     | commit | F6   | Mem[load1]    |
| 2 | no   | FLD F2, 45(R3)     | commit | F2   | Mem[load2]    |
| 3 | no   | FMUL.D F0, F2, F4  | commit | F0   | #2 * Regs[F4] |
| 4 | no   | FSUB.D F8, F6, F2  | commit | F8   | F6 - #2       |
| 5 | Yes  | FDIV.D F10, F0, F6 | Ex40   | F10  |               |
| 6 | Yes  | FADD.D F6, F8, F2  | write  | F6   | #4 + F2       |

|       | Busy | Instruction |
|-------|------|-------------|
| Load1 | no   |             |
| Load2 | no   |             |
| Load3 |      |             |

**Reorder Buffer**

|      | F0 | F2 | F4 | F6  | F8 | F10 | ... | F30 |
|------|----|----|----|-----|----|-----|-----|-----|
| ROB  |    |    |    | #6  |    | #5  |     |     |
| Busy | no | no | no | Yes | no | Yes | ... | no  |



# Tomasulo with Reorder Buffer - Cycle 57

|       | Busy | Op  | Vj            | Vk       | Qj | Qk | Dest | A |
|-------|------|-----|---------------|----------|----|----|------|---|
| Add1  | No   |     |               |          |    |    |      |   |
| Add2  | No   | Add | #4            | Regs[F2] |    |    | #6   |   |
| Add3  | No   |     |               |          |    |    |      |   |
| Mult1 | No   |     |               |          |    |    |      |   |
| Mult2 | No   | Div | #2 * Regs[F4] | Regs[F6] | #3 |    | #5   |   |

Reservation Stations

|   | Busy | Instruction        | Status | Dest | Value         |
|---|------|--------------------|--------|------|---------------|
| 1 | no   | FLD F6, 34(R2)     | commit | F6   | Mem[load1]    |
| 2 | no   | FLD F2, 45(R3)     | commit | F2   | Mem[load2]    |
| 3 | no   | FMUL.D F0, F2, F4  | commit | F0   | #2 * Regs[F4] |
| 4 | no   | FSUB.D F8, F6, F2  | commit | F8   | F6 - #2       |
| 5 | Yes  | FDIV.D F10, F0, F6 | write  | F10  | #3/F6         |
| 6 | Yes  | FADD.D F6, F8, F2  | write  | F6   | #4 + F2       |

|       | Busy | Instruction |
|-------|------|-------------|
| Load1 | no   |             |
| Load2 | no   |             |
| Load3 |      |             |

Reorder Buffer

|      | F0 | F2 | F4 | F6  | F8 | F10 | ... | F30 |
|------|----|----|----|-----|----|-----|-----|-----|
| ROB  |    |    |    | #6  |    | #5  |     |     |
| Busy | no | no | no | Yes | no | Yes | ... | no  |





# Tomasulo with Reorder Buffer - Cycle 58

|       | Busy | Op  | Vj | Vk       | Qj | Qk | Dest | A |
|-------|------|-----|----|----------|----|----|------|---|
| Add1  | No   |     |    |          |    |    |      |   |
| Add2  | No   | Add | #4 | Regs[F2] |    |    | #6   |   |
| Add3  | No   |     |    |          |    |    |      |   |
| Mult1 | No   |     |    |          |    |    |      |   |
| Mult2 | No   |     |    |          |    |    |      |   |

Reservation Stations

|   | Busy | Instruction        | Status | Dest | Value         |
|---|------|--------------------|--------|------|---------------|
| 1 | no   | FLD F6, 34(R2)     | commit | F6   | Mem[load1]    |
| 2 | no   | FLD F2, 45(R3)     | commit | F2   | Mem[load2]    |
| 3 | no   | FMUL.D F0, F2, F4  | commit | F0   | #2 * Regs[F4] |
| 4 | no   | FSUB.D F8, F6, F2  | commit | F8   | F6 - #2       |
| 5 | no   | FDIV.D F10, F0, F6 | commit | F10  | #3/F6         |
| 6 | Yes  | FADD.D F6, F8, F2  | write  | F6   | #4 + F2       |

|       | Busy | Instruction |
|-------|------|-------------|
| Load1 | no   |             |
| Load2 | no   |             |
| Load3 |      |             |

Reorder Buffer

|      | F0 | F2 | F4 | F6  | F8 | F10 | ... | F30 |
|------|----|----|----|-----|----|-----|-----|-----|
| ROB  |    |    |    | #6  |    |     |     |     |
| Busy | no | no | no | Yes | no | no  | ... | no  |



# Tomasulo with Reorder Buffer - Cycle 59

|       | Busy | Op | Vj | Vk | Qj | Qk | Dest | A |
|-------|------|----|----|----|----|----|------|---|
| Add1  | No   |    |    |    |    |    |      |   |
| Add2  | No   |    |    |    |    |    |      |   |
| Add3  | No   |    |    |    |    |    |      |   |
| Mult1 | No   |    |    |    |    |    |      |   |
| Mult2 | No   |    |    |    |    |    |      |   |

Reservation Stations

|   | Busy | Instruction        | Status | Dest | Value         |
|---|------|--------------------|--------|------|---------------|
| 1 | no   | FLD F6, 34(R2)     | commit | F6   | Mem[load1]    |
| 2 | no   | FLD F2, 45(R3)     | commit | F2   | Mem[load2]    |
| 3 | no   | FMUL.D F0, F2, F4  | commit | F0   | #2 * Regs[F4] |
| 4 | no   | FSUB.D F8, F6, F2  | commit | F8   | F6 - #2       |
| 5 | no   | FDIV.D F10, F0, F6 | commit | F10  | #3/F6         |
| 6 | no   | FADD.D F6, F8, F2  | commit | F6   | #4 + F2       |

|       | Busy | Instruction |
|-------|------|-------------|
| Load1 | no   |             |
| Load2 | no   |             |
| Load3 |      |             |

Reorder Buffer

|      | F0 | F2 | F4 | F6 | F8 | F10 | ... | F30 |
|------|----|----|----|----|----|-----|-----|-----|
| ROB  |    |    |    |    |    |     |     |     |
| Busy | no | no | no | no | no | no  | ... | no  |



# Tomasulo with Reorder Buffer - Summary

| Instruction        | Issue | Exec Comp | Writeback | Commit |
|--------------------|-------|-----------|-----------|--------|
| FLD F6, 34(R2)     | 1     | 3         | 4         | 5      |
| FLD F2, 45(R3)     | 2     | 4         | 5         | 6      |
| FMUL.D F0, F2, F4  | 3     | 6-15      | 16        | 17     |
| FSUB.D F8, F6, F2  | 4     | 6-7       | 8         | 18     |
| FDIV.D F10, F0, F6 | 5     | 17-56     | 57        | 58     |
| FADD.D F6, F8, F2  | 6     | 9-10      | 11        | 59     |

- In-order Issue/Commit, Out-of-Order Execution/Writeback

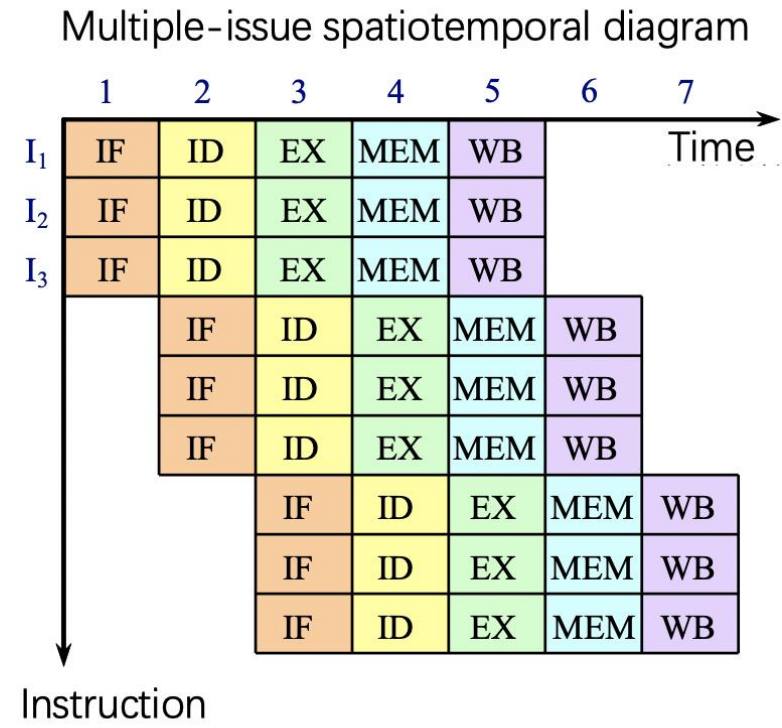
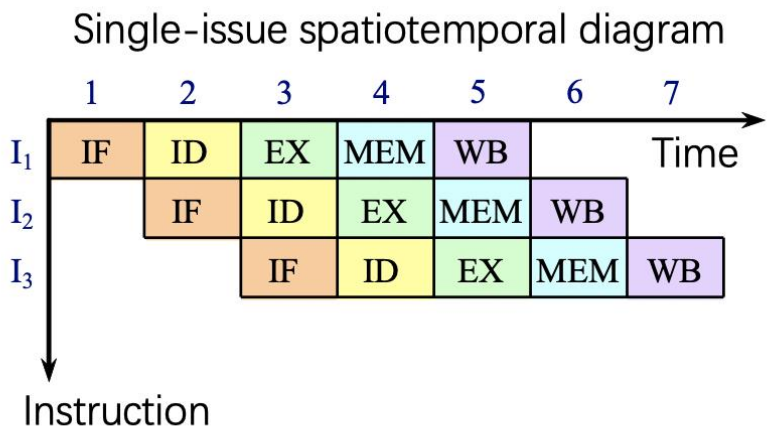


# Hardware-Based Speculation

- Instructions are finished in order according to ROB
- It can be precise exception.
- It is easily extended to integer register and integer function unit.
- But the hardware is too complex.



# Comparison of the spatiotemporal diagrams of instructions executed by single-issue and multiple-issue processors



# Two types of multiple-issue processor

## Superscalar

- The number of instructions which are issued in each clock cycle is **not fixed**. It depends on the specific circumstances of the code. (1-8, with upper limit)
- Suppose this upper limit is **n**, then the processor is called **n-issue**.
- It can be statically scheduled through the compiler, or dynamically scheduled based on the Tomasulo algorithm.
- This method is the most successful method for general computing at present.



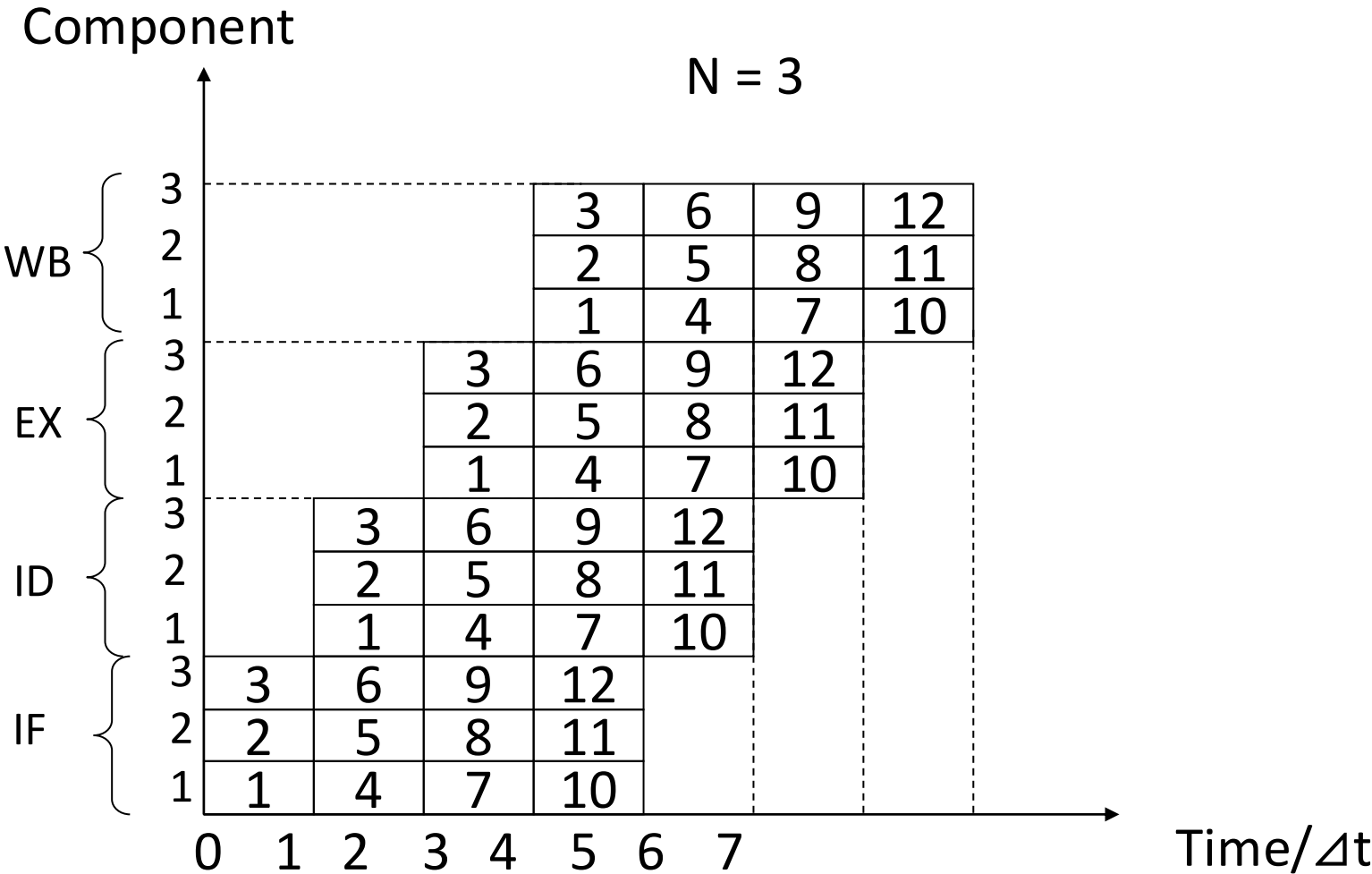
# Two types of multiple-issue processor

## Superscalar

- The number of instructions which are issued in each clock cycle is **not fixed**. It depends on the specific circumstances of the code. (1-8, with upper limit)
- Suppose this upper limit is **n**, then the processor is called **n-issue**.
- It can be statically scheduled through the compiler, or dynamically scheduled based on the Tomasulo algorithm.
- This method is the most successful method for general computing at present.



# Two types of multiple-issue processor





# Two types of multiple-issue processor

## VLIW (Very Long Instruction Word)

- The number of instructions which are issued in each clock cycle is **fixed** (4-16), and these instructions constitute a long instruction or an instruction packet.
- In the instruction packet, the parallelism between instructions is explicitly expressed through instructions.
- Instruction scheduling is done statically by the compiler.
- It has been successfully applied to digital signal processing and multimedia applications.

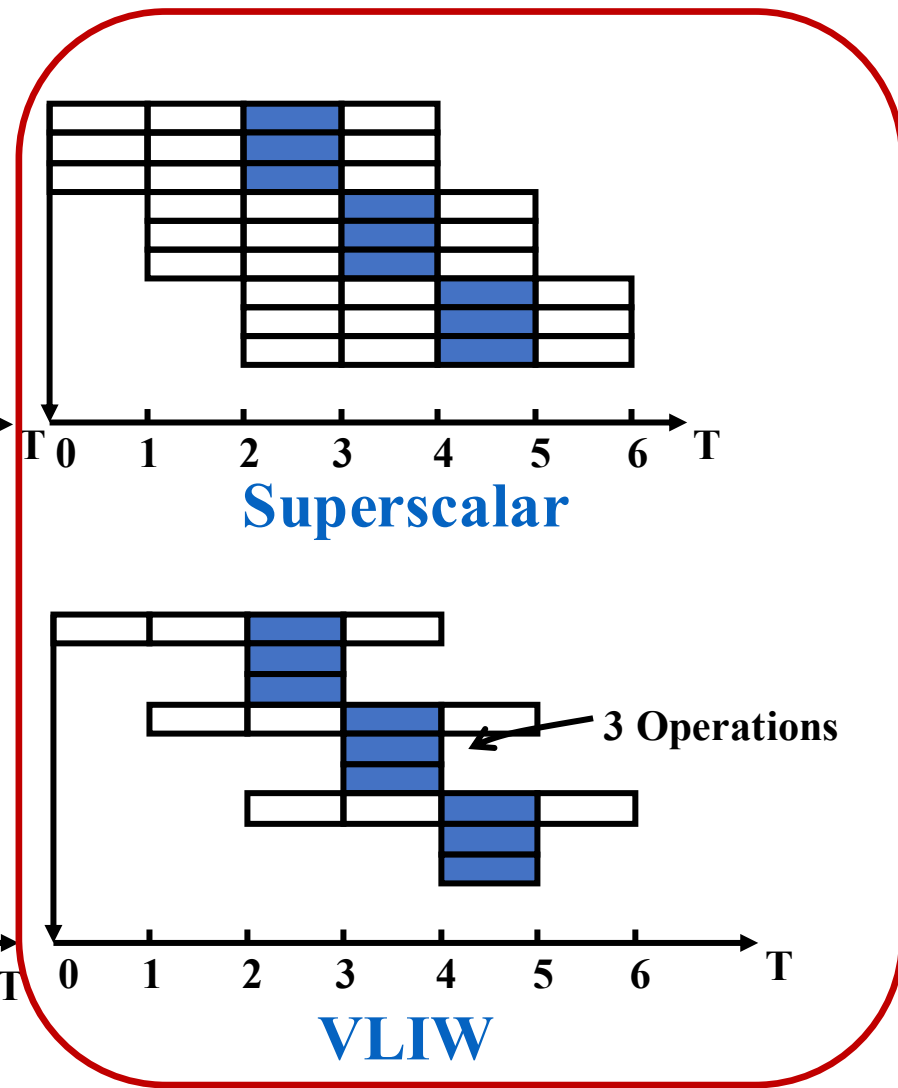
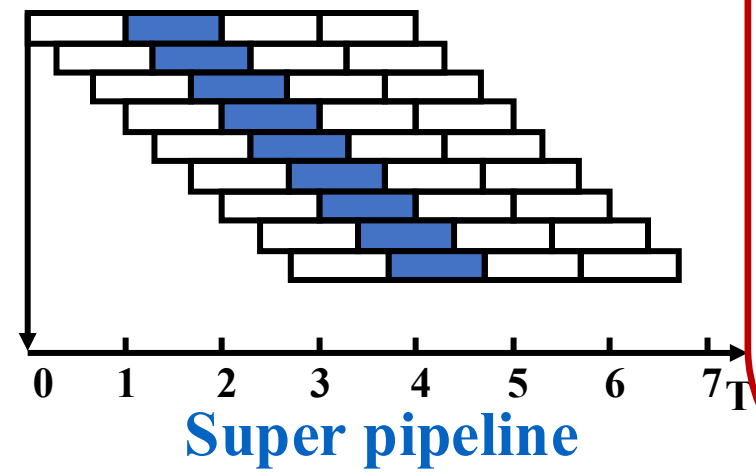
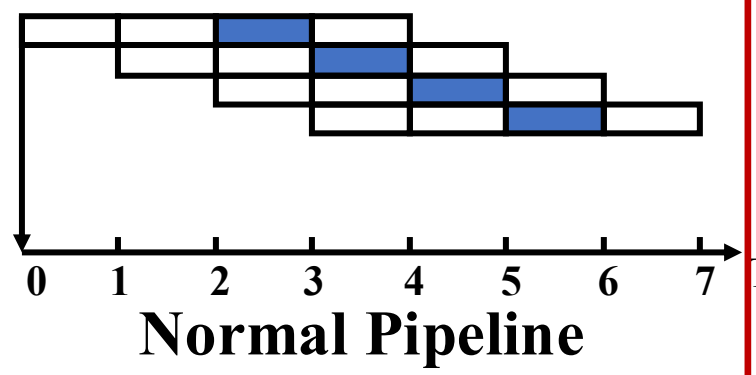


# Superscalar & VLIM

- The superscalar structure is transparent to the programmer, because the processor can detect whether the next instruction can flow out, so there is no need to rearrange instructions to satisfy the issue of instructions.
- Even the code that has not been optimized by the compiler for scheduling and optimization of the superscalar structure or the code generated by the old compiler can run, of course, the running effect will not be very good.
- To achieve good results, one of the methods:
  - Use dynamic superscalar scheduling technology.



# Superscalar & VLIM



## §2.3 Exploiting ILP Using Multiple Issue and Static Scheduling

| Common name               | Issue structure  | Hazard detection   | Scheduling               | Distinguishing characteristic                                       | Examples                                                                |
|---------------------------|------------------|--------------------|--------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------|
| Superscalar (static)      | Dynamic          | Hardware           | Static                   | In-order execution                                                  | Mostly in the embedded space: MIPS and ARM, including the ARM Cortex-A8 |
| Superscalar (dynamic)     | Dynamic          | Hardware           | Dynamic                  | Some out-of-order execution, but no speculation                     | None at the present                                                     |
| Superscalar (speculative) | Dynamic          | Hardware           | Dynamic with speculation | Out-of-order execution with speculation                             | Intel Core i3, i5, i7; AMD Phenom; IBM Power 7                          |
| VLIW/LIW                  | Static           | Primarily software | Static                   | All hazards determined and indicated by compiler (often implicitly) | Most examples are in signal processing, such as the TI C6x              |
| EPIC                      | Primarily static | Primarily software | Mostly static            | All hazards determined and indicated explicitly by the compiler     | Itanium                                                                 |



# Multi-issue technology based on static scheduling

- In a typical superscalar processor, **1 to 8 instructions** can be issued per clock cycle.
- Instructions flow out in order, and conflict detection is performed when they flow out.
  - In the current sequence of instructions, there is no data conflict or Close conflict.

Example: A statically scheduled superscalar processor with **4 issues**

- In the instruction fetch stage, the pipeline will receive 1 to 4 instructions (called issue packets) from the instruction fetch component.
- In one clock cycle, all of these instructions may be able to flow out, or only a part of them may flow out.



# Multi-issue technology based on static scheduling

The outgoing component detects structural conflicts or data conflicts.

- Generally implemented in two stages:
  - **The first stage:** Carry out the conflict detection in the outgoing package, and select the instructions that can be outflowed initially.
  - **The second stage:** Check whether the selected instruction conflicts with the instruction being executed.

How does the MIPS processor achieve superscalar?

- Assumption: Two instructions flow out every clock cycle:
  - **1 integer instruction + 1 floating-point operation instruction**
- Among them, **load** instructions, **store** instructions, and branch instructions are classified as integer instructions.



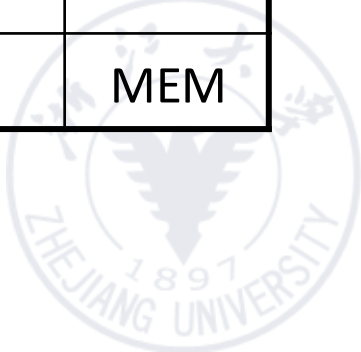
# Multi-issue technology based on static scheduling

Claim:

- Fetch two instructions (64 bits) at the same time and decode two instructions (64 bits).
- The processing of instructions includes the following steps:
  - Fetch two instructions from Cache.
  - Determine which instructions can flow out (0~2 instructions).
  - Send them to the corresponding functional components.
- The execution process of instructions in a multiple-issue superscalar pipeline
  - Assumption: All floating-point instructions are addition instructions, and their execution time is two clock cycles.
  - For simplicity, integer instructions are always placed before floating-point instructions in the figure below.



| Type                       | Pipeline work bench |    |    |     |     |     |     |     |
|----------------------------|---------------------|----|----|-----|-----|-----|-----|-----|
| Integer Instruction        | IF                  | ID | EX | MEM | WB  |     |     |     |
| Floating-Point Instruction | IF                  | ID | EX | EX  | MEM | WB  |     |     |
| Integer Instruction        |                     | IF | ID | EX  | MEM | WB  |     |     |
| Floating-Point Instruction |                     | IF | ID | EX  | EX  | MEM | WB  |     |
| Integer Instruction        |                     |    | IF | ID  | EX  | MEM | WB  |     |
| Floating-Point Instruction |                     |    | IF | ID  | EX  | EX  | MEM | WB  |
| Integer Instruction        |                     |    |    | IF  | ID  | EX  | MEM | WB  |
| Floating-Point Instruction |                     |    |    | IF  | ID  | EX  | EX  | MEM |





# Multi-issue technology based on static scheduling

- With the parallel outflow method of "**1 integer instruction + 1 floating point instruction**", the amount of hardware that needs to be increased is small.
- Floating-point **load** or floating-point **store** instructions will use integer parts, which will increase access conflicts to floating point registers.
  - Add a read/write port for floating-point registers.
- Since the number of instructions in the pipeline has doubled, the directional path has to be increased.



# Multi-issue technology based on dynamic scheduling

- Extended Tomasulo algorithm: supports two-way superscalar
  - Two instructions are issued every clock cycle;
  - One is an integer instruction and the other is a floating-point instruction.
- Use a relatively simple method:
  - Instructions flow to the RS in order, otherwise the program semantics will be destroyed.
  - Separate the table structure used for integers from the table structure used for floating-point, and process them separately, so that one floating-point instruction and one integer instruction can be sent to their respective reservation stations at the same time.



# Multi-issue technology based on dynamic scheduling

- For the RISC-V pipeline that uses the Tomasulo algorithm and multi-issue technology, consider the following simple loop execution. This program adds the scalar in X2 to each element of a vector.

```
Loop:    LD      X2, 0 (X1)      // X2=array element
         ADDI    X2, X2, 1       // increment X2
         SD      X2, 0 (X1)      // store result
         ADDI    X1, X1, 8       // increment pointer by 8
                                   // (each data occupies 8 bytes)
         BNE     X2, X3, Loop    // branch if not last
```



# Multi-issue technology based on dynamic scheduling

Now make the following assumptions:

- One integer instruction and one floating-point instruction can flow out every clock cycle, even if they are related.
- There is an integer component for integer ALU operations and address calculations; and for each type of floating-point operation, there is an independent pipelined floating-point functional component.
- The instruction flow and the write result each take one clock cycle.
- It has a dynamic branch prediction component and an independent functional component for calculating branch conditions.
- Branch instructions flowed out separately, no delayed branch was used, but branch prediction was perfect. Before the branch instruction is completed, its subsequent instructions can only be fetched and flowed out, but cannot be executed.



# Multi-issue technology based on dynamic scheduling

Because the write result occupies one clock cycle, the delay in generating the result is: one cycle for integer operations, two cycles for load, and three cycles for floating-point addition.

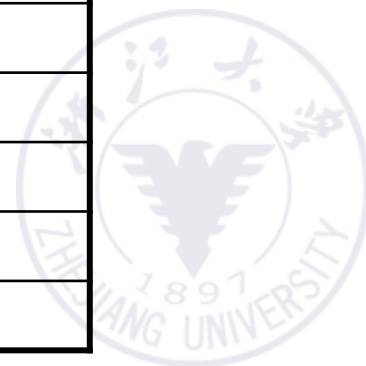
List the issue of each instruction in the first **three** loops of the program, start execution, and write the results to the **CDB**.

## Solution:

When execution, the loop will be dynamically unrolled, and two instructions will issue whenever possible. For ease of analysis, the time when the memory fetch occurs is listed in the table. The running result is shown in the figure below.



| Iteration number | Instruction      | IS | EX | MEM | Write CDB | Explanation                 |
|------------------|------------------|----|----|-----|-----------|-----------------------------|
| 1                | LD X2, 0 (X1)    | 1  | 2  | 3   | 4         | Issue the first instruction |
| 1                | ADDI X2, X2, 1   | 1  | 5  |     | 6         | Wait for LD                 |
| 1                | SD X2, 0 (X1)    | 2  | 3  | 7   |           | Wait for ADDI               |
| 1                | ADDI X1, X1, 8   | 2  | 3  |     | 4         | Execute directly            |
| 1                | BNE X2, X3, Loop | 3  | 7  |     |           | Wait for ADDI               |
| 2                | LD X2, 0 (X1)    | 4  | 8  | 9   | 10        | Wait for BNE                |
| 2                | ADDI X2, X2, 1   | 4  | 11 |     | 12        | Wait for LD                 |
| 2                | SD X2, 0 (X1)    | 5  | 9  | 13  |           | Wait for ADDI               |
| 2                | ADDI X1, X1, 8   | 5  | 8  |     | 9         | Wait for BNE                |
| 2                | BNE X2, X3, Loop | 6  | 13 |     |           | Wait for ADDI               |
| 3                | LD X2, 0 (X1)    | 7  | 14 | 15  | 16        | Wait for BNE                |
| 3                | ADDI X2, X2, 1   | 7  | 17 |     | 18        | Wait for LD                 |
| 3                | SD X2, 0 (X1)    | 8  | 13 | 19  |           | Wait for ADDI               |
| 3                | ADDI X1, X1, 8   | 8  | 14 |     | 15        | Wait for BNE                |
| 3                | BNE X2, X3, Loop | 9  | 19 |     |           | Wait for ADDI               |



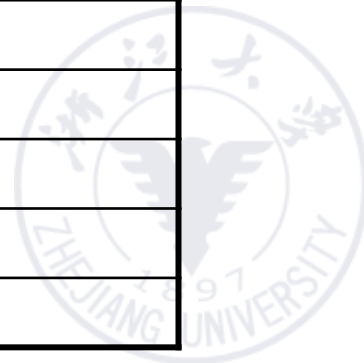
# Multi-issue technology based on dynamic scheduling

It can be seen from the table:

- Although the outflow rate of instructions is relatively high, the execution efficiency is not very high.
  - A total of 15 instructions were executed in 19 clock cycles.
  - The average command execution speed is  $15/19=0.79$  per clock cycle.
- The reason is there are data-dependent branches and the ALU components have become a bottleneck.
- Solution: Add an adder to separate the ALU function from the address calculation function.



| Iteration number | Instruction      | IS | EX | MEM | Write CDB | Commit | Explanation                 |
|------------------|------------------|----|----|-----|-----------|--------|-----------------------------|
| 1                | LD X2, 0 (X1)    | 1  | 2  | 3   | 4         | 5      | Issue the first instruction |
| 1                | ADDI X2, X2, 1   | 1  | 5  |     | 6         | 7      | Wait for LD                 |
| 1                | SD X2, 0 (X1)    | 2  | 3  |     |           | 7      | Wait for ADDI               |
| 1                | ADDI X1, X1, 8   | 2  | 3  |     | 4         | 8      | Commit in order             |
| 1                | BNE X2, X3, Loop | 3  | 7  |     |           | 8      | Wait for ADDI               |
| 2                | LD X2, 0 (X1)    | 4  | 5  | 6   | 7         | 9      | No execute delay            |
| 2                | ADDI X2, X2, 1   | 4  | 8  |     | 9         | 10     | Wait for LD                 |
| 2                | SD X2, 0 (X1)    | 5  | 6  |     |           | 10     | Wait for ADDI               |
| 2                | ADDI X1, X1, 8   | 5  | 6  |     | 7         | 11     | Commit in order             |
| 2                | BNE X2, X3, Loop | 6  | 10 |     |           | 11     | Wait for ADDI               |
| 3                | LD X2, 0 (X1)    | 7  | 8  | 9   | 10        | 12     | Earliest possible           |
| 3                | ADDI X2, X2, 1   | 7  | 11 |     | 12        | 13     | Wait for LD                 |
| 3                | SD X2, 0 (X1)    | 8  | 9  |     |           | 13     | Wait for ADDI               |
| 3                | ADDI X1, X1, 8   | 8  | 9  |     | 10        | 14     | Executes earlier            |
| 3                | BNE X2, X3, Loop | 9  | 13 |     |           | 14     | Wait for ADDI               |





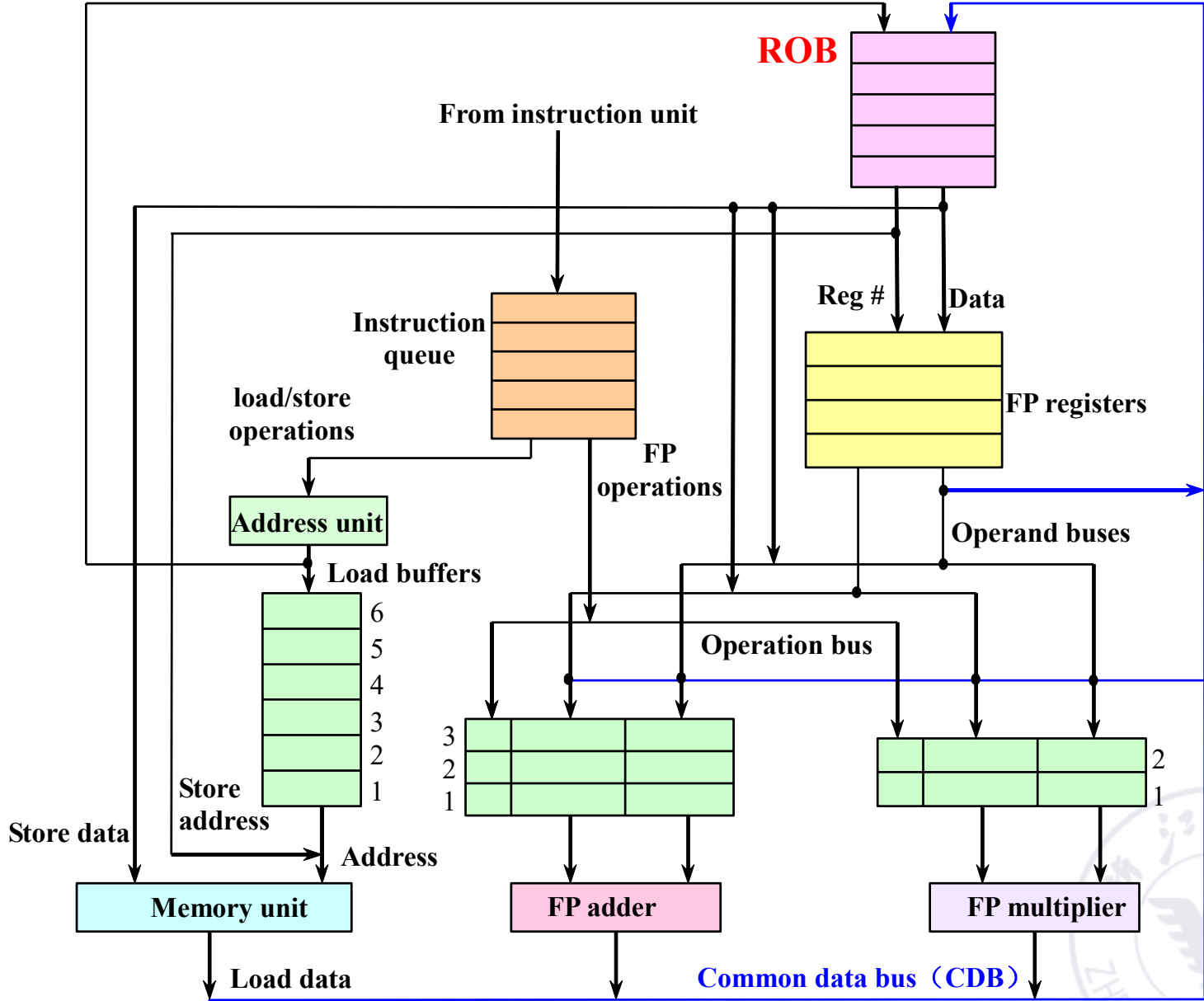
# Multi-issue technology based on dynamic scheduling

It can be seen from this example:

- In this case, where a branch, can be a critical performance limiter, speculation helps significantly.
- The completion rate on the nonspeculative pipeline is falling behind the issue rate rapidly, the nonspeculative pipeline will stall when a few more iterations are issued.
- This example clearly shows how speculation can be advantageous when there are data-dependent branches, which otherwise would limit performance.
- This advantage depends on accurate branch prediction.
- Incorrect speculation does not improve performance; in fact, it typically harms performance and dramatically lowers energy efficiency.



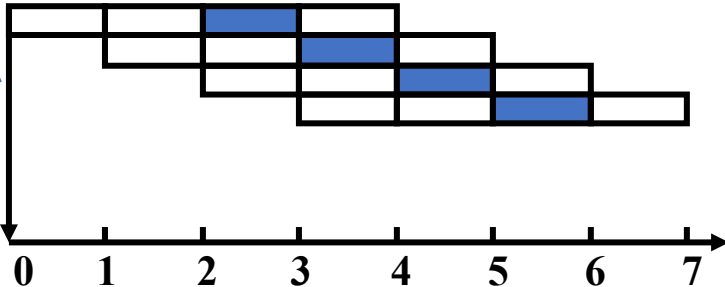
The basic structure of a FP unit using Tomasulo's algorithm and extended to handle speculation.



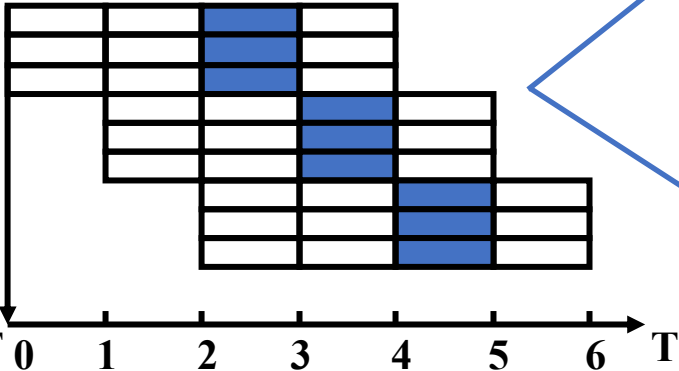
# Superscalar & VLIM

Pipeline based on static scheduling

Pipeline based on dynamic scheduling



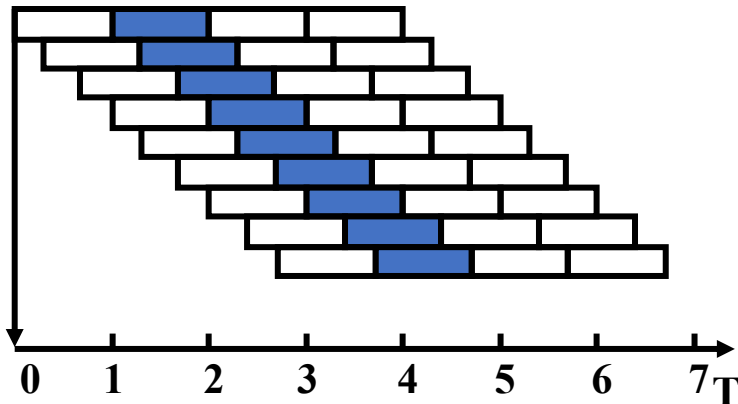
Normal Pipeline



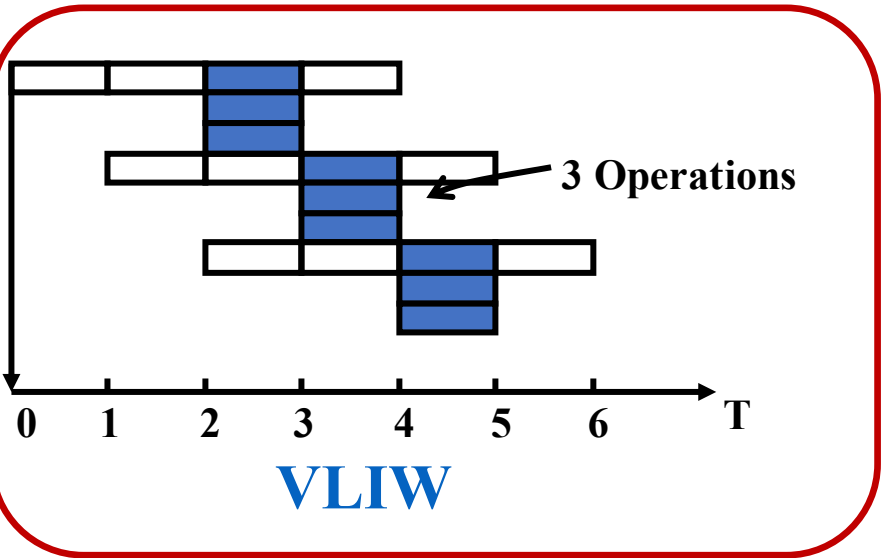
Superscalar

Multi-issue technology based on static scheduling

Multi-issue technology based on dynamic scheduling



Super pipeline



VLIW

3 Operations

static scheduling

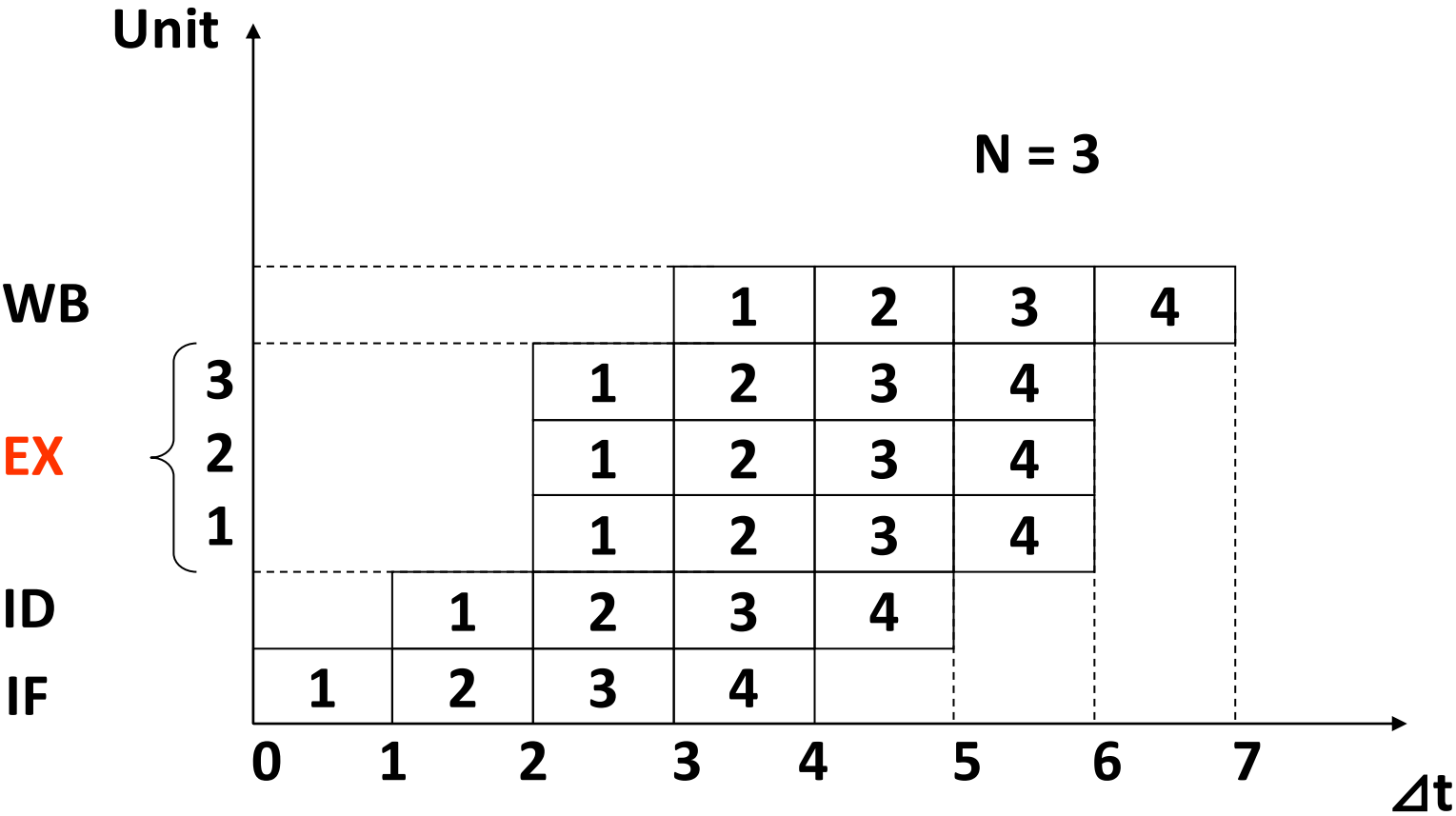


# Very long instruction word technology(VLIW)

- Assemble multiple instructions that can be executed in parallel into a very long instruction (more than 100 bits to hundreds of bits).
- Set up multiple features.
- The instruction word is divided into several fields, and each field is called an **operation slot**, which directly and independently controls a functional unit.
- In the VLIW processor, all processing and instruction arrangement are completed by the compiler.
- At compile time, multiple unrelated or unrelated operations that can be executed in parallel are combined to **form a very long instruction word** with **multiple operation segments**.



VLIW



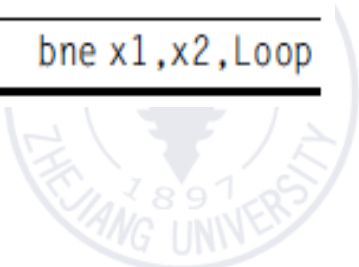
# Very long instruction word technology(VLIW)

Show an unrolled version of the loop  $X[i] = x[i] + s$ .

```
Loop: fld      f0,0(x1)      //f0=array element
      fadd.d   f4,f0,f2      //add scalar in f2
      fsd     f4,0(x1)      //store result
      addi    x1,x1,-8       //decrement pointer
                               //8 bytes (per DW)
      bne     x1,x2,Loop     //branch x1≠x2
```

```
Loop: fld      f0,0(x1)
      fld      f6,-8(x1)
      fld      f0,-16(x1)
      fld      f14,-24(x1)
      fadd.d   f4,f0,f2
      fadd.d   f8,f6,f2
      fadd.d   f12,f0,f2
      fadd.d   f16,f14,f2
      fsd     f4,0(x1)
      fsd     f8,-8(x1)
      fsd     f12,16(x1)
      fsd     f16,8(x1)
      addi    x1,x1,-32
      bne     x1,x2,Loop
```

| Memory reference 1 | Memory reference 2 | FP operation 1     | FP operation 2    | Integer operation/branch |
|--------------------|--------------------|--------------------|-------------------|--------------------------|
| fld f0,0(x1)       | fld f6,-8(x1)      |                    |                   |                          |
| fld f10,-16(x1)    | fld f14,-24(x1)    |                    |                   |                          |
| fld f18,-32(x1)    | fld f22,-40(x1)    | fadd.d f4,f0,f2    | fadd.d f8,f6,f2   |                          |
| fld f26,-48(x1)    |                    | fadd.d f12,f0,f2   | fadd.d f16,f14,f2 |                          |
|                    |                    | fadd.d f20,f18,f2  | fadd.d f24,f22,f2 |                          |
| fsd f4,0(x1)       | fsd f8,-8(x1)      | fadd.d f28,f26,f24 |                   |                          |
| fsd f12,-16(x1)    | fsd f16,-24(x1)    |                    |                   | addi x1,x1,-56           |
| fsd f20,24(x1)     | fsd f24,16(x1)     |                    |                   |                          |
| fsd f28,8(x1)      |                    |                    |                   | bne x1,x2,Loop           |



# Very long instruction word technology(VLIW)

Some problems with VLIW

- Program code length increased
  - A large number of loop unrolling to improve parallelism.
  - The operation slot in the instruction word cannot always be filled.
  - Solution: Use the method of command sharing the immediate digital field, or use the method of command compression storage, transfer to Cache or expansion during decoding.
- Lockstep mechanism
  - When any operating part is paused, the entire processor must be paused.
- Machine code incompatibility



# Very long instruction word technology(VLIW)

What are the limitations of the instruction multi-flow processor?

- Mainly affected by the following three aspects:
  - Instruction-level parallelism inherent in the program.
  - Difficulties in hardware implementation.
  - Technical limitations inherent in superscalar and super long instruction word processors.

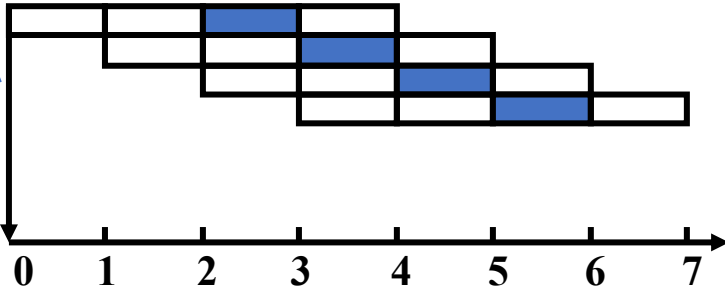




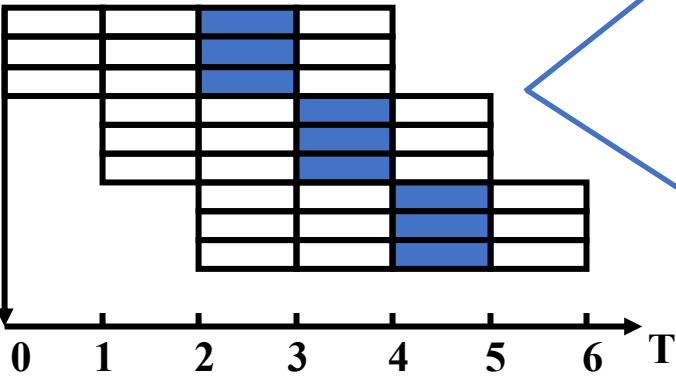
# Superscalar & VLIM

Pipeline based on static scheduling

Pipeline based on dynamic scheduling



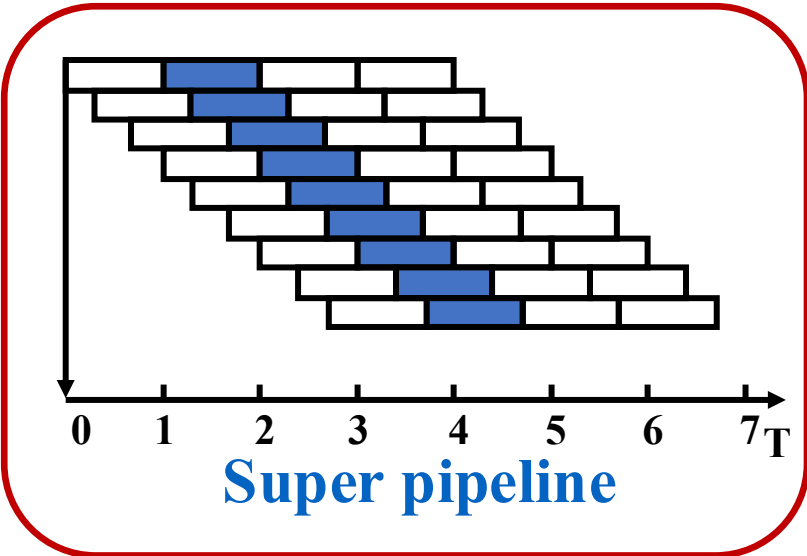
Normal Pipeline



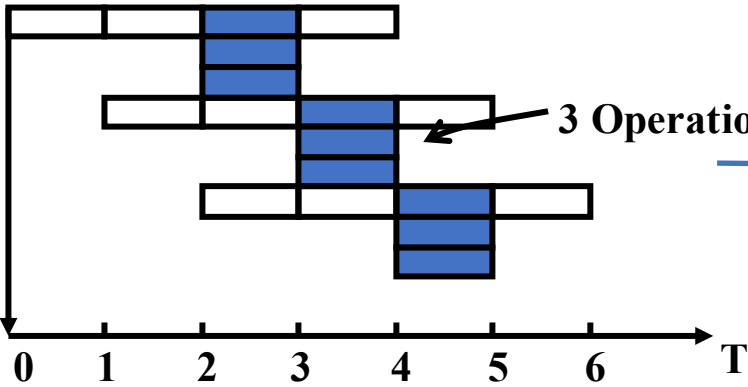
Superscalar

Multi-issue technology based on static scheduling

Multi-issue technology based on dynamic scheduling



Super pipeline



VLIW

3 Operations

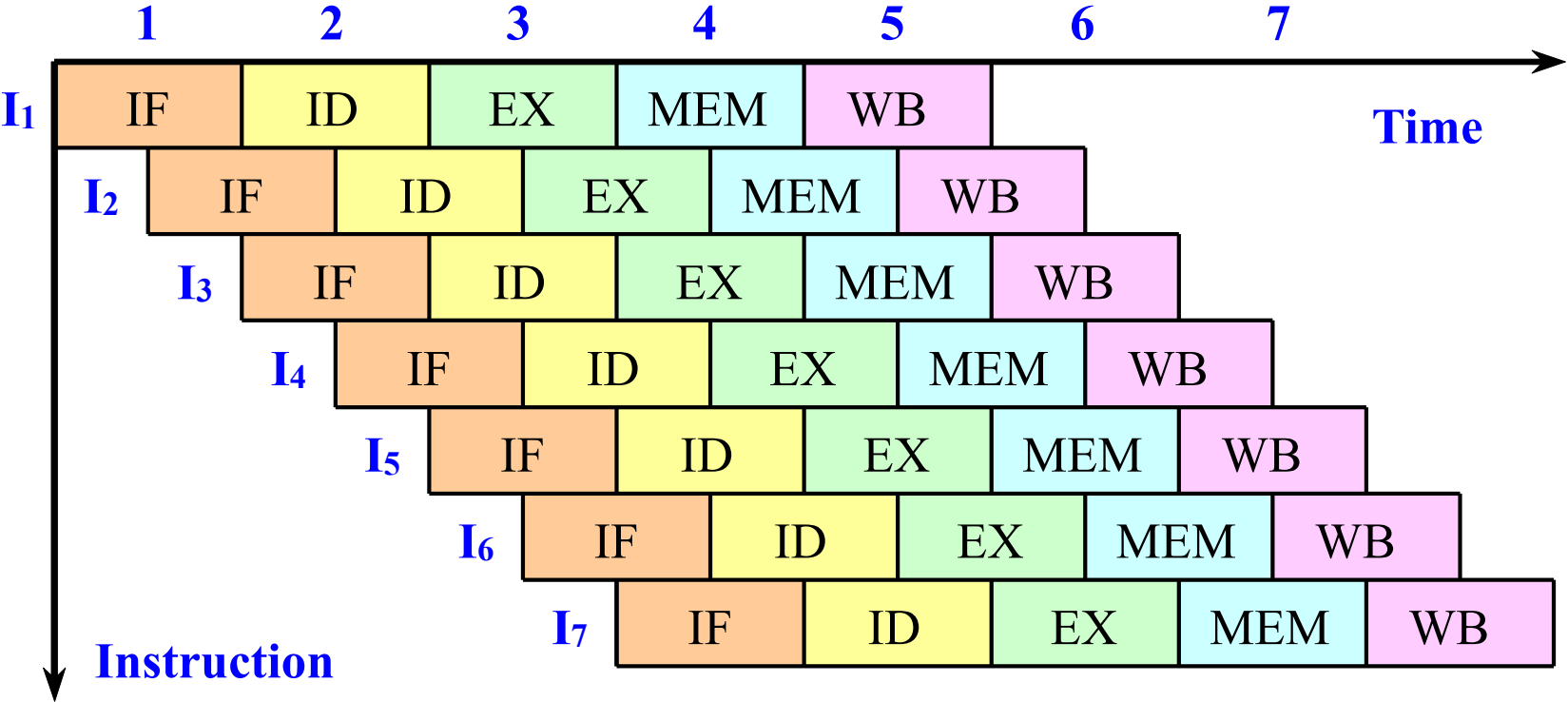
static scheduling

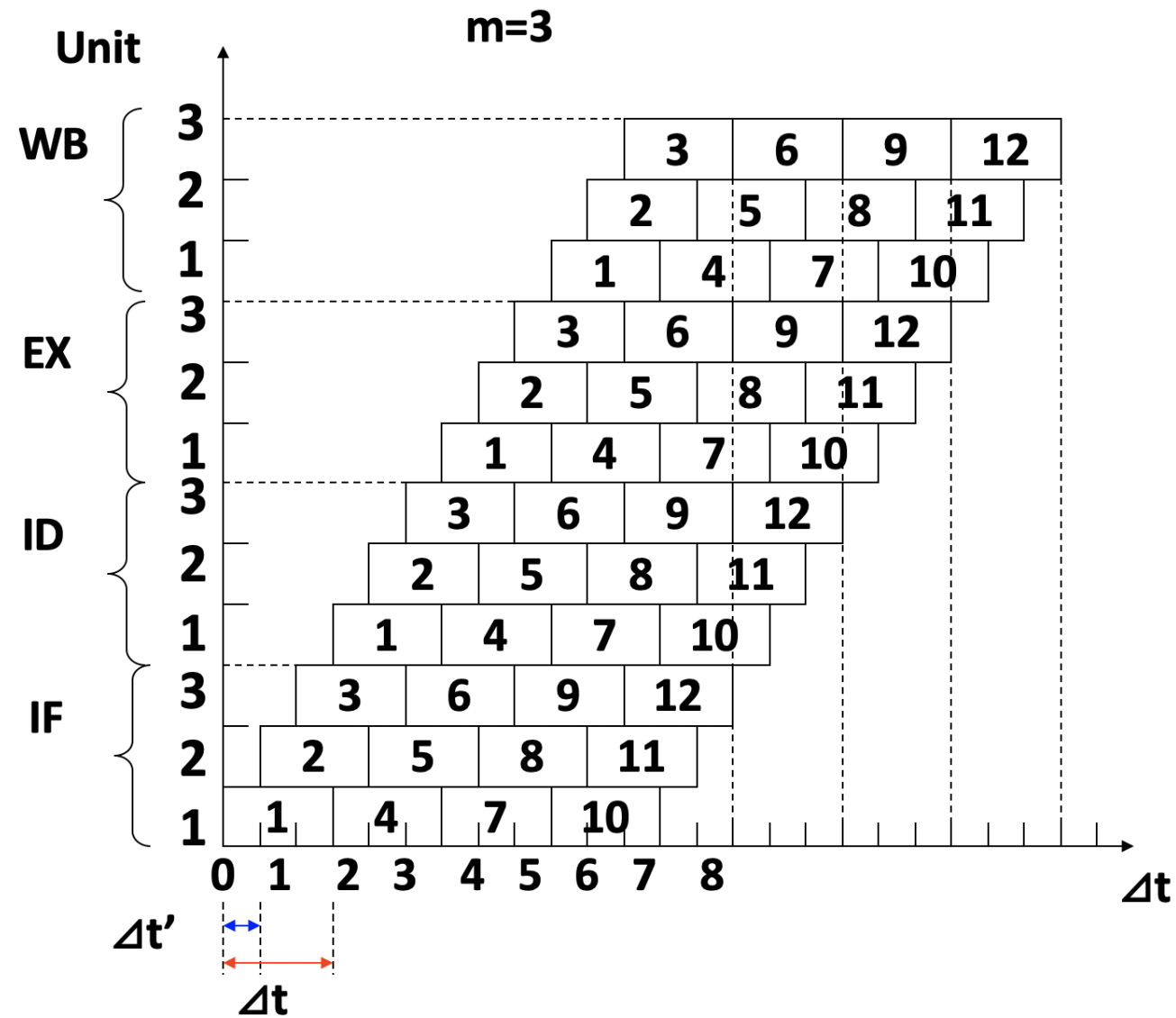


# Super pipelined

- Each pipeline stage is further subdivided, so that multiple instructions can be time-shared in one clock cycle. This kind of processor is called a super-pipelined processor.
- For a super-pipelined computer that can flow out  $n$  instructions per clock cycle, these  $n$  instructions are not flowed out at the same time, but one instruction is flowed out every  $1/n$  clock cycle.
  - In fact, the pipeline cycle of the super-pipeline computer is  $1/n$  clock cycles.
- The time-space diagram of a super-pipelined computer that issues two instructions in time-sharing every clock cycle.





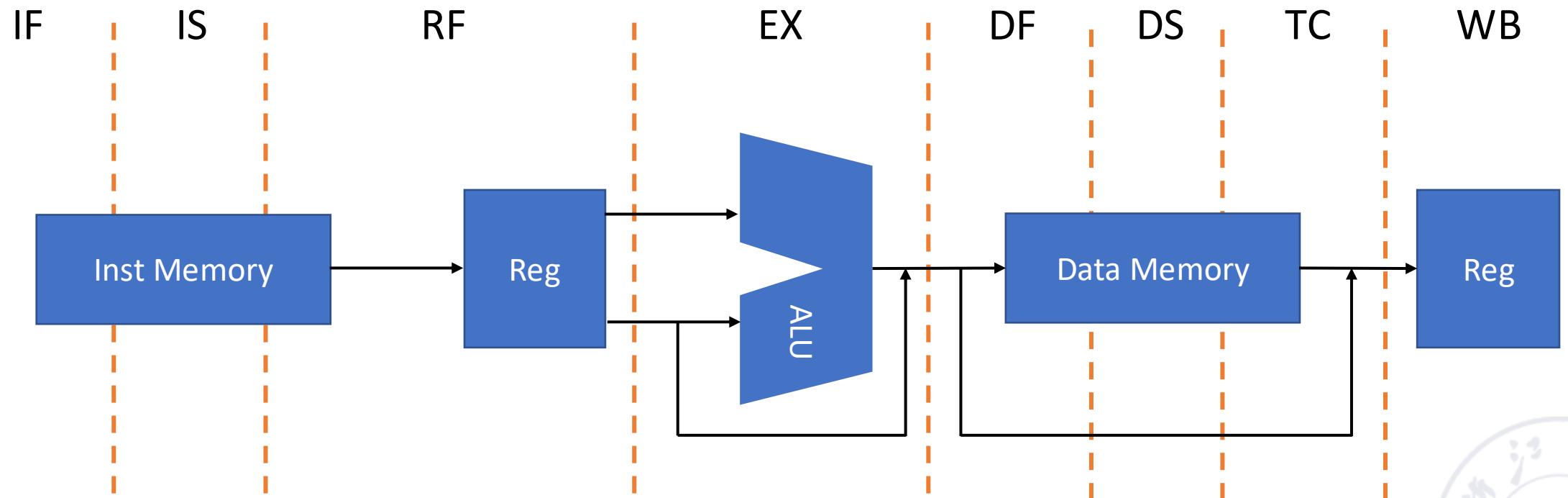


# Superpipelining processor

- A pipeline processor with 8 or more instruction pipeline stages is called a superpipelining processor.
- Typical superpipelining processor: SGI's MIPS series R4000
  - There are 2 Caches in the R4000 microprocessor chip:
    - Instruction Cache and Data Cache
    - The capacity is 8 KB
    - The data width of each Cache is 64 b
  - R4000's core processing components: integer components
    - A  $32 \times 32$  bit general register bank
    - An arithmetic logic unit (ALU)
    - A dedicated multiplication/division unit



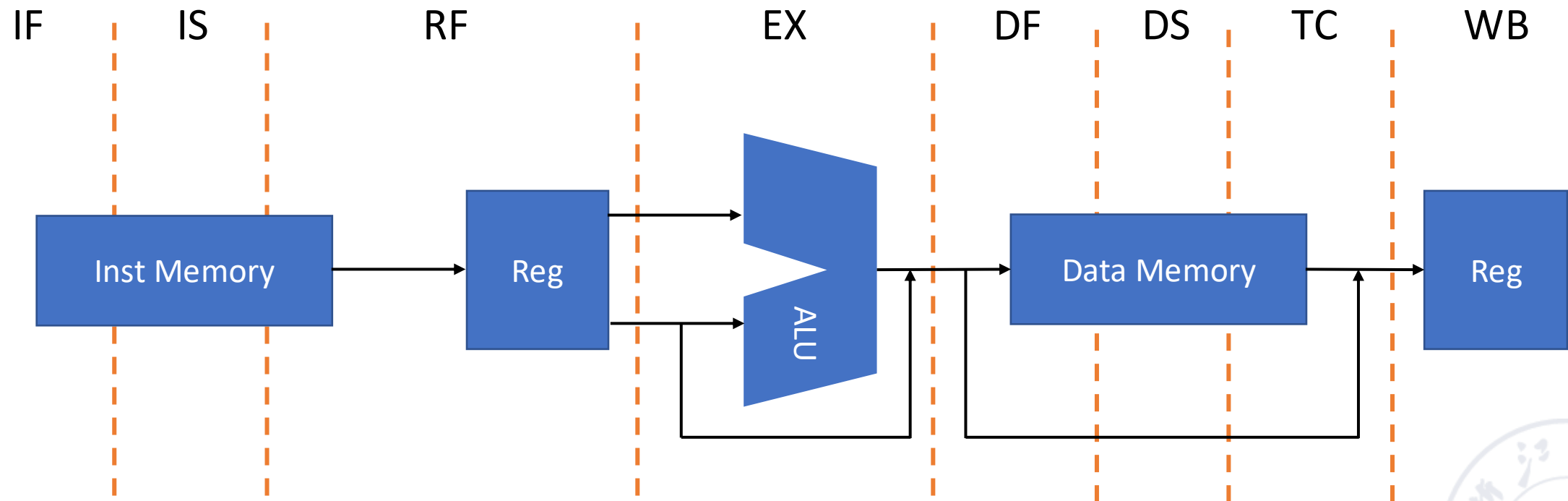
# R4000 Pipeline Structure



- IF—First half of instruction fetch; PC selection actually happens here, together with initiation of instruction cache access.



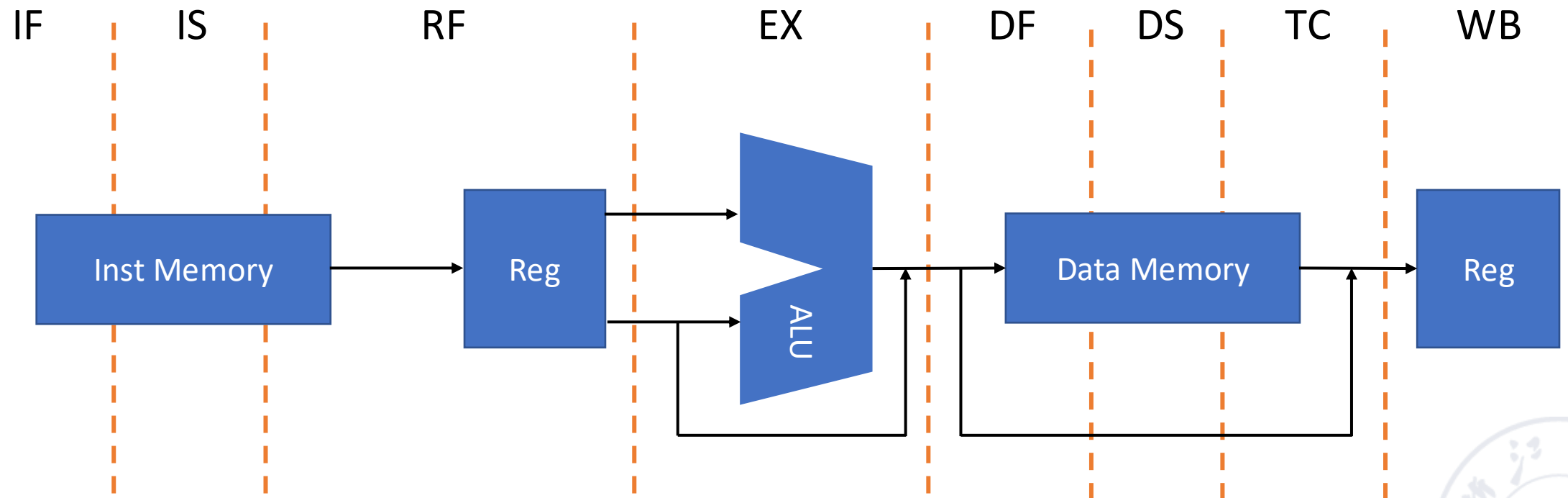
# R4000 Pipeline Structure



- IS—Second half of instruction fetch, complete instruction cache access.



# R4000 Pipeline Structure

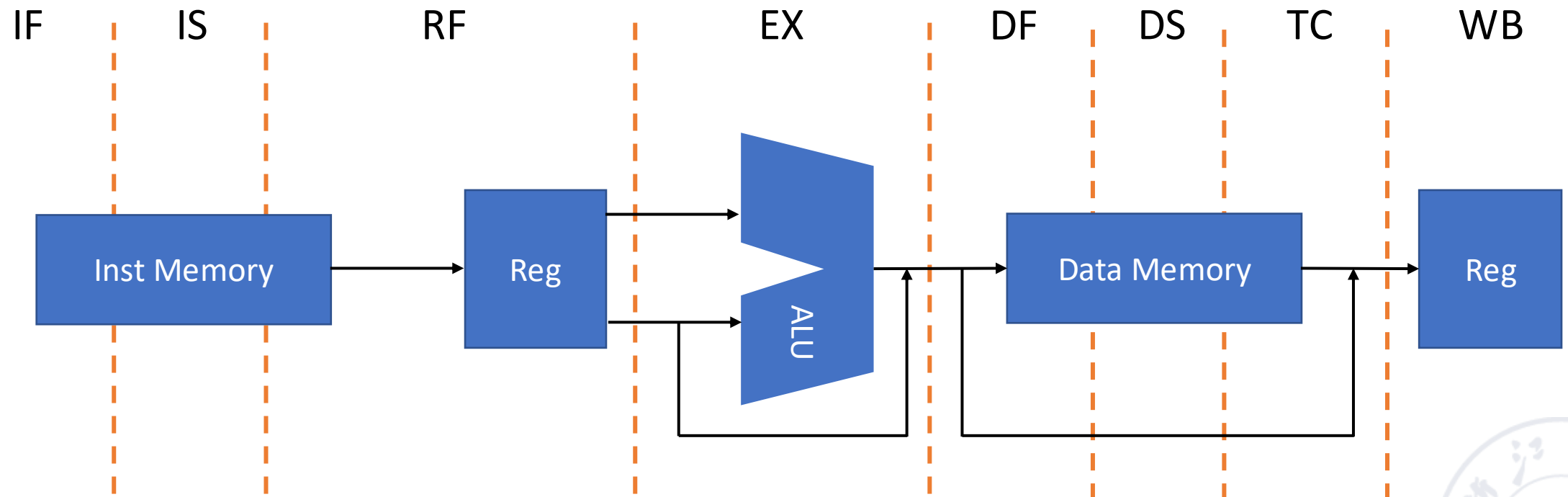


- RF—Instruction decode and register fetch, hazard checking, and instruction cache hit detection.





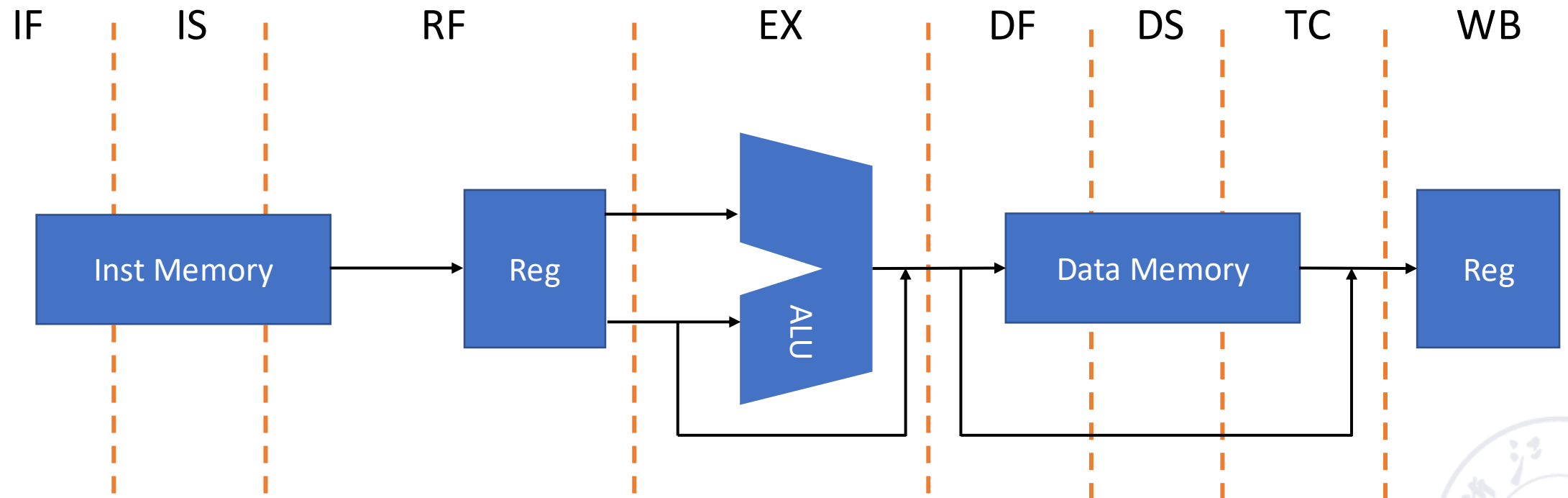
# R4000 Pipeline Structure



- EX—Execution, which includes effective address calculation, ALU operation, and branch-target computation and condition evaluation.



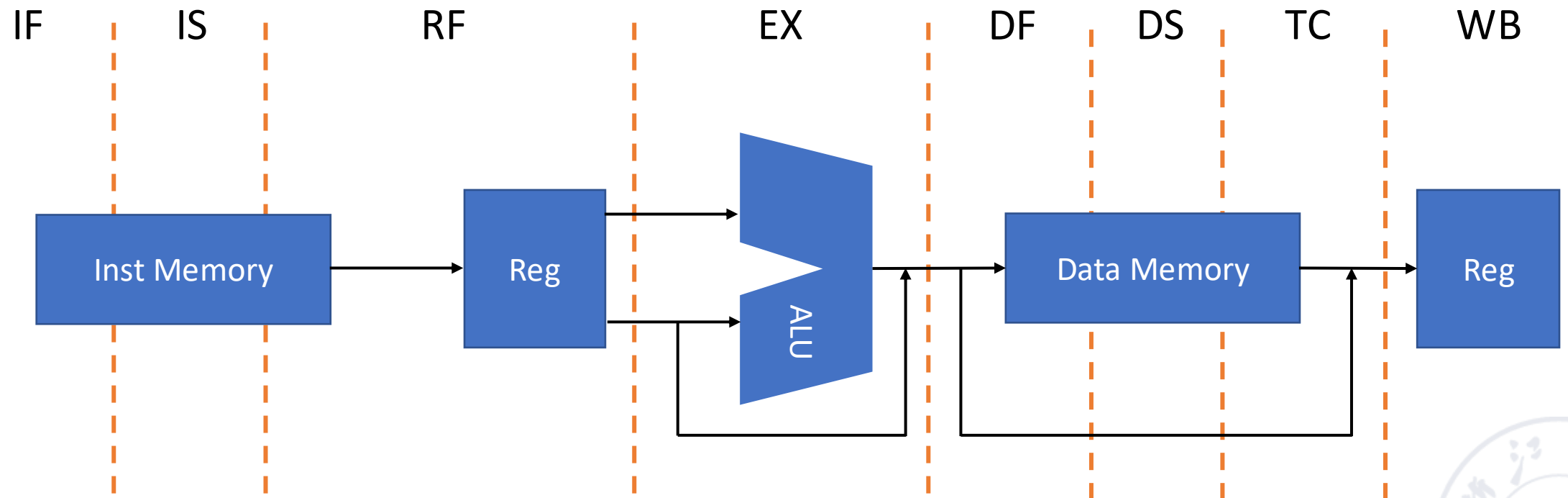
# R4000 Pipeline Structure



- DF—Data fetch, first half of data cache access.
- DS—Second half of data fetch, completion of data cache access.



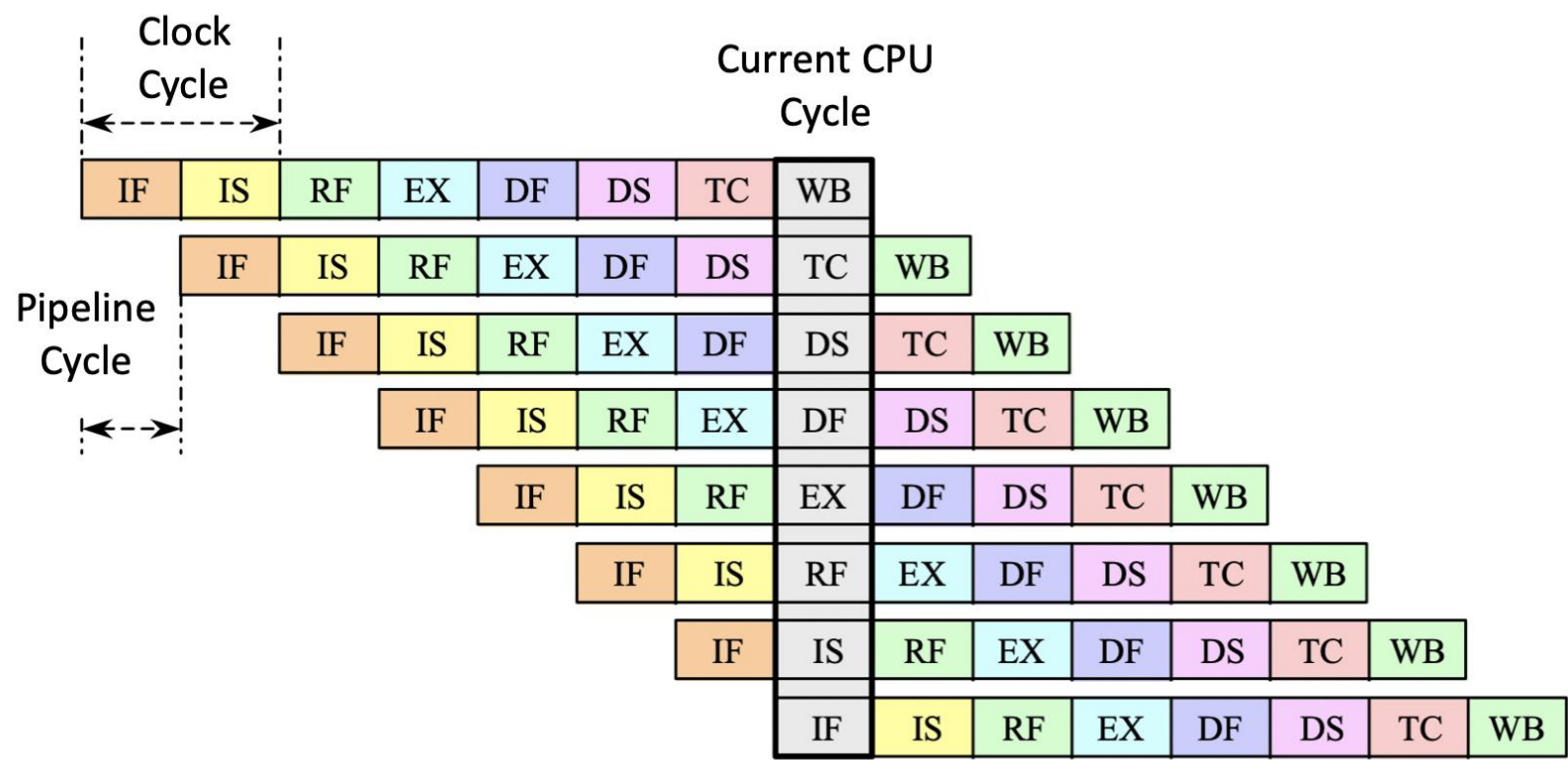
# R4000 Pipeline Structure



- TC—Tag check, to determine whether the data cache access hit.
- WB—Write-back for loads and register-register operations.



# Spatiotemporal Diagram of MIPS R4000 Pipeline



# Two Clock Cycles for Load Delay

